

Supporting information for “The risks of dichotomising ordinal outcome measures: A spatial analysis of self-rated health in Western Europe by age and education”

Miguel Ángel Beltrán-Sánchez^a; Miguel Ángel Martínez-Beneito^a; Terje Eikemo^b;
Tiago Alexandre Alcobia Pereira^c; Sara Martino^c; Andrea Riebler^c

^aDepartment of Statistics and Operational Research, University of Valencia, Burjassot, Spain

^bDepartment of Sociology and Political Science, Norwegian University of Science and Technology, Trondheim, Norway

^cDepartment of Mathematical Sciences, Norwegian University of Science and Technology, Trondheim, Norway

Required packages

```
# # IMPORTANT:
# # Next line should be run once to install the latest development version from GitHub, enabling
# ↪ the use of HMC with dcar_leroux
# remotes::install_github("nimble-dev/nimble", subdir = "packages/nimble")

# install.packages("pacman", dep = TRUE)

pacman::p_load(sf, spdep, readxl, abind, ggplot2, RColorBrewer, patchwork,
               nimble, nimbleHMC, MCMCvis, scales, install = FALSE)

# # Loading functions for calling Nimble
#
# ↪ source("https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNimble.0.2.R")
# load.leroux()
```

European Social Survey (ESS) data loading

```
# ----- #
# Data source: European Social Survey (ESS)
#
# To replicate the analysis, please download the dataset from:
# https://ess.sikt.no/en/datafile/242aaa39-3bbb-40f5-98bf-bfb1ce53d8ef
#
# Access to the ESS data requires a free user registration.
# Once downloaded, extract the .zip file and place the extracted
# CSV file in the "data" folder before running this script.
# ----- #

# ESS data
ESSData <- read.csv(file.path("..", "data", "ESS11e04_1.csv"))

# Western Europe data
```

```

country <- "west"
ESSData <- ESSData[ESSData$cntry == "AT" | ESSData$cntry == "BE" |
  ESSData$cntry == "CH" | ESSData$cntry == "DE" |
  ESSData$cntry == "ES" | ESSData$cntry == "FR" |
  ESSData$cntry == "IT" | ESSData$cntry == "NL" |
  ESSData$cntry == "PT", ]

ESSData <- ESSData[ESSData$region != "ES63" & ESSData$region != "ES64" & ESSData$region != "ES70"
  &
    !grepl("^FRY|^FRZ", ESSData$region) & ESSData$region != "FRM0" &
    ESSData$region != "PT20" & ESSData$region != "PT30", ]

# Definition of the categorical covariates
# Age groups: 1 = (14, 34]; 2 = (34, 54]; 3 = (54, 100]
ESSData$agecut <- cut(ESSData$agea, c(14, 34, 54, 100))
levels(ESSData$agecut) <- c("15-34", "35-54", "55...")
# Education groups: 1 = (-1, 225]; 2 = (225, 500]; 3 = (500, 1000]
ESSData$educut <- cut(ESSData$edulvlb, c(-1, 225, 500, 1000))
levels(ESSData$educut) <- c("edulow", "edumid", "eduhigh")

# ESS cleaned
ESS <- ESSData[, c("health", "region", "gndr", "agecut", "educut",
  "dweight", "pspwght", "anweight", "cntry",
  "domain", "psu", "stratum", "prob")]

# Removing NA rows for age and education groups for simplicity
ESS <- ESS[!is.na(ESS$agecut), ]
ESS <- ESS[!is.na(ESS$educut), ]
str(ESS)

```

```

## 'data.frame':    17053 obs. of  13 variables:
## $ health : int  3 2 1 3 2 1 2 2 2 2 ...
## $ region : chr  "AT31" "AT22" "AT33" "AT31" ...
## $ gndr : int  1 2 2 2 1 2 2 2 2 2 ...
## $ agecut : Factor w/ 3 levels "15-34","35-54",...: 3 1 2 3 3 3 3 3 2 3 ...
## $ educut : Factor w/ 3 levels "edulow","edumid",...: 2 2 3 2 2 2 1 3 2 1 ...
## $ dweight : num  1.185 0.61 1.392 0.556 0.723 ...
## $ pspwght : num  0.393 0.325 4 0.176 1.061 ...
## $ anweight: num  0.13 0.1076 1.3237 0.0583 0.3511 ...
## $ cntry : chr  "AT" "AT" "AT" "AT" ...
## $ domain : int  2 1 2 1 2 2 2 2 2 2 ...
## $ psu : int  317 128 418 295 344 373 86 3 108 314 ...
## $ stratum : int  107 69 18 101 115 7 58 38 62 105 ...
## $ prob : num  0.000579 0.001124 0.000493 0.001233 0.000949 ...

```

```

# Transforming gender and region into factors
ESS$gndr <- factor(ESS$gndr, labels = c("Man", "Woman"))

# Western Europe
ESS$region <- factor(ESS$region,
  levels = c("AT11", "AT12", "AT13", "AT21", "AT22", "AT31", "AT32",
    "AT33", "AT34", "BE10", "BE21", "BE22", "BE23", "BE24",
    "BE25", "BE31", "BE32", "BE33", "BE34", "BE35", "CH01",
    "CH02", "CH03", "CH04", "CH05", "CH06", "CH07", "DE1",
    "DE2", "DE3", "DE4", "DE5", "DE6", "DE7", "DE8", "DE9",
    "DEA", "DEB", "DEC", "DED", "DEE", "DEF", "DEG", "ES11",

```

```

"ES12", "ES13", "ES21", "ES22", "ES23", "ES24", "ES30",
"ES41", "ES42", "ES43", "ES51", "ES52", "ES53", "ES61",
"ES62", "FR10", "FRB0", "FRC1", "FRC2", "FRD1", "FRD2",
"FRE1", "FRE2", "FRF1", "FRF2", "FRF3", "FRG0", "FRH0",
"FRI1", "FRI2", "FRI3", "FRJ1", "FRJ2", "FRK1", "FRK2",
"FRLO", "ITC", "ITF", "ITG", "ITH", "ITI", "NL11", "NL12",
"NL13", "NL21", "NL22", "NL23", "NL31", "NL32", "NL33",
"NL34", "NL41", "NL42", "PT11", "PT15", "PT16", "PT17",
"PT18"))

# # Design weight
# weight <- ESS$dweight/mean(ESS$dweight)
# # Post-stratification weight including design weight
# weight <- ESS$pspwght/mean(ESS$pspwght)
# Analysis weight
weight <- ESS$anweight/mean(ESS$anweight)

# Gender of each respondent
gnd <- as.numeric(ESS$gndr)
# Age group of each respondent
age <- as.numeric(ESS$agecut)
# Education group of each respondent
edu <- as.numeric(ESS$educut)
# NUTS of each respondent
nuts <- as.numeric(ESS$region)

```

health - Subjective general health

```

# How is your health in general? Would you say it is...
# 1 = Very good, 2 = Good, 3 = Fair, 4 = Bad, 5 = Very bad,
# 7 = Refusal, 8 = Don't know, 9 = No answer
table(ESS$health)

```

```

##
##      1      2      3      4      5      7      8
## 3832 7761 4244 1046  154   11    5

```

```

y <- as.numeric(ESS$health)
y[y > 5] <- NA
variable <- "health"
y_labels <- c("Very good", "Good", "Fair", "Bad", "Very bad")

# survey: data frame containing the variables collected for each respondent (ESS)
# - nuts: small-area (NUTS1) from 1 to number of areas
# - gnd: 1 = Man, 2 = Woman
# - age: 1 = (14, 34]; 2 = (34, 54]; 3 = (54, 100]
# - edu: 1 = (-1, 225]; 2 = (225, 500]; 3 = (500, 1000]
# - weight: design/post-stratification/analysis weight
survey <- data.frame("y" = y, "nuts" = nuts, "gnd" = gnd,
                     "age" = age, "edu" = edu, "weight" = weight)

```

Spatial neighbourhood structure

```
# Cartography of Europe
# Source: Eurostat GISCO - NUTS 2021 regions
# https://gisco-services.ec.europa.eu/distribution/v2/nuts/download/
cartography <- st_read(file.path("../data", "NUTS_RG_03M_2021_4326.shp"))
```

```
## Reading layer `NUTS_RG_03M_2021_4326' from data source
##   `/home/bsmangel/Escritorio/Trabajo/Doctorado/Investigo/Proyecto/nimble/dichotomisation/data/NUTS_R
##   using driver `ESRI Shapefile'
## Simple feature collection with 2010 features and 8 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -63.15176 ymin: -21.38696 xmax: 55.83509 ymax: 80.83427
## Geodetic CRS:   WGS 84
```

```
cartography <- cartography[order(cartography$NUTS_ID), ]

# Western Europe
cartography <- cartography[(cartography$LEVL_CODE == 2 &
                             cartography$CNTR_CODE %in% c("AT", "BE", "CH", "ES",
                                                           "FR", "NL", "PT")) |
                             (cartography$LEVL_CODE == 1 &
                              cartography$CNTR_CODE %in% c("DE", "IT")), ]
cartography <- cartography[cartography$NUTS_ID != "ES63" &
                           cartography$NUTS_ID != "ES64" &
                           cartography$NUTS_ID != "ES70" &
                           !grepl("^FRY|^FRZ", cartography$NUTS_ID) &
                           cartography$NUTS_ID != "FRM0" &
                           cartography$NUTS_ID != "PT20" &
                           cartography$NUTS_ID != "PT30", ]
cartography <- cartography[order(cartography$NUTS_ID), ]

# Neighborhood structure by contiguity
Neigh <- poly2nb(cartography)

# Manually add adjacency between Cataluña (ES51), Comunitat Valenciana (ES52)
# and Illes Balears (ES53) to avoid treating Illes Balears as isolated.
Neigh[[which(cartography$NUTS_ID == "ES53")]] <- unique(c(Neigh[[which(cartography$NUTS_ID ==
  ↪ "ES53")]],
                                                         which(cartography$NUTS_ID == "ES51"))))
Neigh[[which(cartography$NUTS_ID == "ES53")]] <- unique(c(Neigh[[which(cartography$NUTS_ID ==
  ↪ "ES53")]],
                                                         which(cartography$NUTS_ID == "ES52"))))

Neigh[[which(cartography$NUTS_ID == "ES51")]] <- unique(c(Neigh[[which(cartography$NUTS_ID ==
  ↪ "ES51")]],
                                                         which(cartography$NUTS_ID == "ES53"))))
Neigh[[which(cartography$NUTS_ID == "ES52")]] <- unique(c(Neigh[[which(cartography$NUTS_ID ==
  ↪ "ES52")]],
                                                         which(cartography$NUTS_ID == "ES53"))))

# Adjacency matrix
W <- nb2mat(Neigh, style = "B", zero.policy = TRUE)
# D - W matrix
Q <- diag(apply(W, 1, sum)) - W
# Number of areas
```

```

NNUTS <- nrow(W)
# Number of neighbors of each area
nadj <- apply(W, 1, sum)
# Neighbors of each area
map <- unlist(apply(W, 1, function(x) which(x != 0)))
# Sum of all the neighbor numbers of all areas
nadj.tot <- length(map)
# Cumulative sums of the number of neighbors of each area
index <- c(0, cumsum(nadj))
# All the neighborhoods j ~ i where i < j
from.to <- cbind(rep(1:NNUTS, times = nadj), map); colnames(from.to) <- c("from", "to")
from.to <- from.to[which(from.to[, 1] < from.to[, 2]), ]
NDist <- nrow(from.to)
# Eigenvalues of D - W
Lambda <- eigen(Q)$values

plot_nuts_neighbours <- function(cartography, nuts_id, nuts_col = "red",
                                neigh_col = "pink", base_col = "grey90",
                                border_col = "black") {

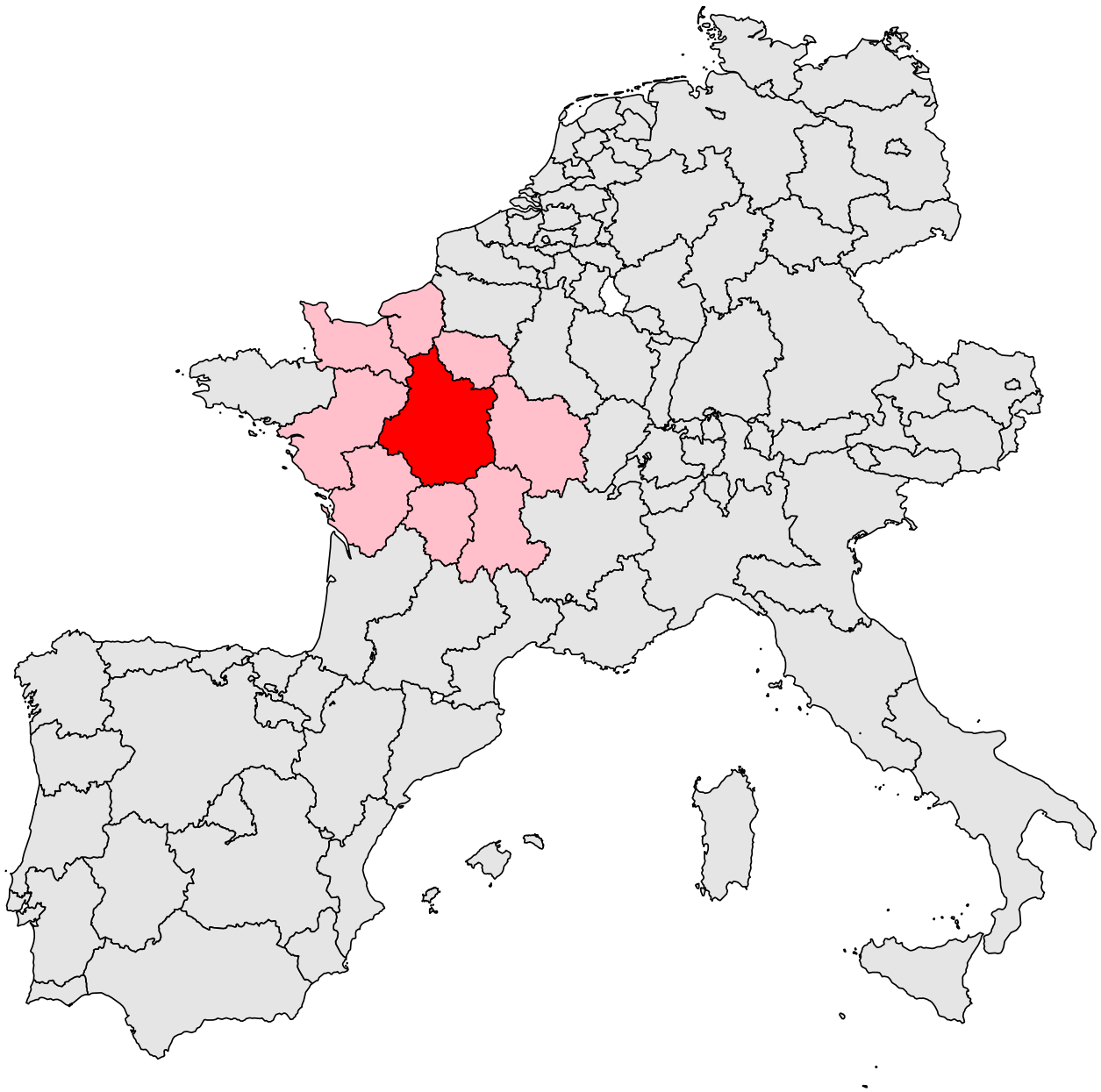
  i <- which(cartography$NUTS_ID == nuts_id)
  fill_col <- rep(base_col, nrow(cartography))
  fill_col[i] <- nuts_col
  fill_col[Neigh[[i]]] <- neigh_col

  plot(st_geometry(cartography), col = fill_col, border = border_col,
       main = paste("NUTS:", nuts_id))
}

# Plot selected regions and their neighbouring areas
plot_nuts_neighbours(cartography, nuts_id = "FRB0")

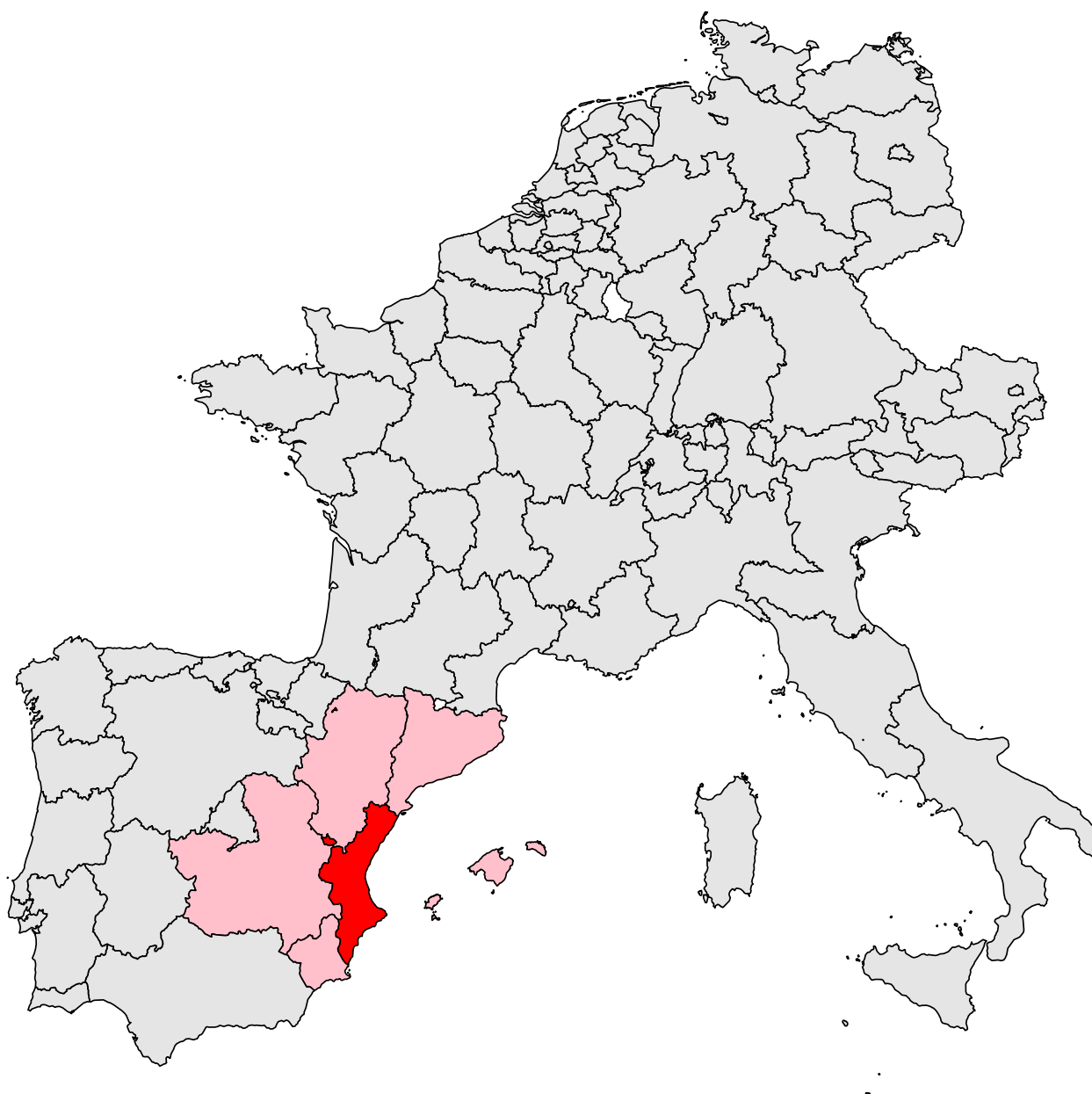
```

NUTS: FRB0



```
plot_nuts_neighbours(cartography, nuts_id = "ES52")
```

NUTS: ES52



```

cartography$NAME_LATN[cartography$NUTS_ID == "AT13"] <- "Vienna"
cartography$NAME_LATN[cartography$NUTS_ID == "BE10"] <- "Brussels-Capital Region"
cartography$NAME_LATN[cartography$NUTS_ID == "CH02"] <- "Espace Mittelland"
cartography$NAME_LATN[cartography$NUTS_ID == "DE3"] <- "Berlin"
cartography$NAME_LATN[cartography$NUTS_ID == "ES30"] <- "Community of Madrid"
cartography$NAME_LATN[cartography$NUTS_ID == "FR10"] <- "Ile-de-France"
cartography$NAME_LATN[cartography$NUTS_ID == "IT1"] <- "Central Italy"
cartography$NAME_LATN[cartography$NUTS_ID == "NL32"] <- "North Holland"
cartography$NAME_LATN[cartography$NUTS_ID == "PT17"] <- "Lisbon Metropolitan Area"

```

Descriptive spatial analysis

```
# Gender selection
Gender <- 1
Gender_Cat <- c("Man", "Woman")[Gender]
gender <- tolower(Gender_Cat)

ordinal_survey <- survey[survey$gnd == Gender, ]
y <- as.numeric(ordinal_survey$y)
y[y > 5] <- NA
table(y)/sum(table(y))
```

```
## y
##          1          2          3          4          5
## 0.240232443 0.469831850 0.228363007 0.053659743 0.007912957
```

```
nuts_all <- factor(ordinal_survey$nuts, levels = 1:NNUTS)
table(nuts_all)
```

```
## nuts_all
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
## 44 199 161 45 156 179 66 82 52 57 168 86 112 93 92 20 60 56 23 26
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
## 143 174 84 96 101 71 20 159 189 32 42 0 19 94 14 109 307 46 12 65
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
## 34 52 30 71 7 14 44 27 4 17 121 66 32 28 119 90 19 146 27 122
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 48 27 17 8 29 57 28 29 16 37 66 58 44 10 33 27 46 23 95 48
## 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
## 406 287 138 235 249 26 32 32 66 119 16 64 153 144 22 108 52 221 24 139
## 101 102
## 165 29
```

```
cartography$y_mean_ordinal <- as.numeric(by(y, nuts_all, mean, na.rm = TRUE))

bernoulli1_survey <- survey[survey$gnd == Gender, ]
y <- as.numeric(bernoulli1_survey$y)
y[y > 5] <- NA
y[y == 1 | y == 2] <- 0
y[y == 3 | y == 4 | y == 5] <- 1
table(y)/sum(table(y))
```

```
## y
##          0          1
## 0.7100643 0.2899357
```

```
nuts_all <- factor(bernoulli1_survey$nuts, levels = 1:NNUTS)
table(nuts_all)
```

```
## nuts_all
##  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
## 44 199 161 45 156 179 66 82 52 57 168 86 112 93 92 20 60 56 23 26
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
## 143 174 84 96 101 71 20 159 189 32 42 0 19 94 14 109 307 46 12 65
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
```



```
## 34 52 30 71 7 14 44 27 4 17 121 66 32 28 119 90 19 146 27 122
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 48 27 17 8 29 57 28 29 16 37 66 58 44 10 33 27 46 23 95 48
## 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
## 406 287 138 235 249 26 32 32 66 119 16 64 153 144 22 108 52 221 24 139
## 101 102
## 165 29
```

```
cartography$y_mean_bernoulli1 <- as.numeric(by(y, nuts_all, mean, na.rm = TRUE))

bernoulli2_survey <- survey[survey$gnd == Gender, ]
y <- as.numeric(bernoulli2_survey$y)
y[y > 5] <- NA
y[y == 1 | y == 2 | y == 3] <- 0
y[y == 4 | y == 5] <- 1
table(y)/sum(table(y))
```

```
## y
##      0      1
## 0.9384273 0.0615727
```

```
nuts_all <- factor(bernoulli2_survey$nuts, levels = 1:NNUTS)
table(nuts_all)
```

```
## nuts_all
## 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
## 44 199 161 45 156 179 66 82 52 57 168 86 112 93 92 20 60 56 23 26
## 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
## 143 174 84 96 101 71 20 159 189 32 42 0 19 94 14 109 307 46 12 65
## 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60
## 34 52 30 71 7 14 44 27 4 17 121 66 32 28 119 90 19 146 27 122
## 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80
## 48 27 17 8 29 57 28 29 16 37 66 58 44 10 33 27 46 23 95 48
## 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
## 406 287 138 235 249 26 32 32 66 119 16 64 153 144 22 108 52 221 24 139
## 101 102
## 165 29
```

```
cartography$y_mean_bernoulli2 <- as.numeric(by(y, nuts_all, mean, na.rm = TRUE))

# Ordinal
limit <- c(min(cartography$y_mean_ordinal, na.rm = TRUE),
           max(cartography$y_mean_ordinal, na.rm = TRUE))
p_y_mean_ordinal <- ggplot(cartography) +
  geom_sf(aes(fill = y_mean_ordinal, color = "grey30", linewidth = 0.1) +
    scale_fill_gradientn(colours = brewer.pal(9, "Purples"),
                        limits = c(limit[1], limit[2]),
                        name = NULL)) + ggtitle("Ordinal") +
  theme_void() + theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12))

# Bernoulli (01)
limit <- c(min(cartography$y_mean_bernoulli1, na.rm = TRUE),
           max(cartography$y_mean_bernoulli1, na.rm = TRUE))
p_y_mean_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = y_mean_bernoulli1, color = "grey30", linewidth = 0.1) +
    scale_fill_gradientn(colours = brewer.pal(9, "Oranges"),
```

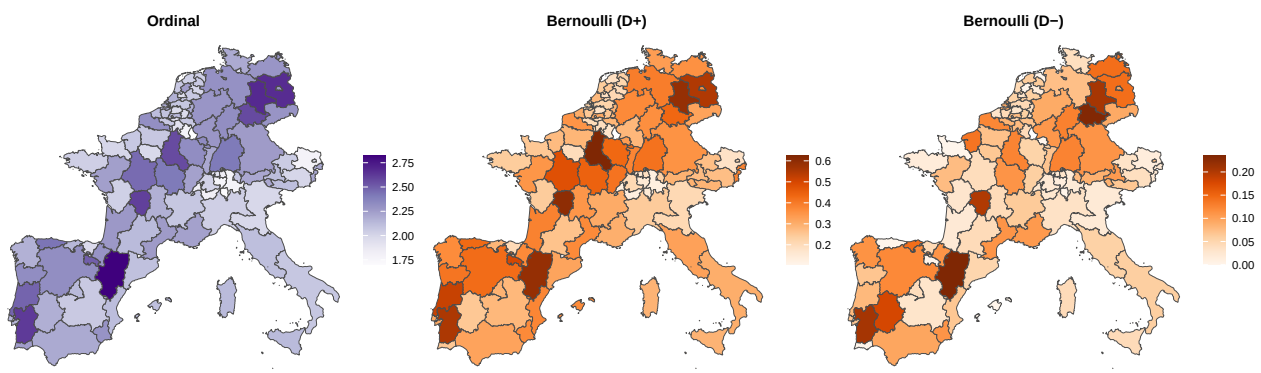
```

        limits = c(limit[1], limit[2]),
        name = NULL) + ggtitle("Bernoulli (D+)") +
theme_void() + theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12))

# Bernoulli (D2)
limit <- c(min(cartography$y_mean_bernoulli2, na.rm = TRUE),
          max(cartography$y_mean_bernoulli2, na.rm = TRUE))
p_y_mean_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = y_mean_bernoulli2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Oranges"),
                      limits = c(limit[1], limit[2]),
                      name = NULL) + ggtitle("Bernoulli (D-)") +
  theme_void() + theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12))

(p_y_mean_ordinal + p_y_mean_bernoulli1 + p_y_mean_bernoulli2)

```



Data preparation and gender filtering

```

# Gender selection
Gender <- 1
survey <- survey[survey$gnd == Gender, ]

# Number of respondents
NResp <- nrow(survey)
# Vector of zeros for the zero-trick
zero <- rep(0, NResp)

# NUTS of each respondent
nuts <- survey$nuts
# Age group of each respondent
age <- survey$age
# Education group of each respondent
edu <- survey$edu
# Sampling weight
weight <- survey$weight

# Response variable
y <- survey$y
# Number of levels
NCats <- length(table(y))
# Vector of ones for Dirichlet prior
ones <- rep(1, NCats)

```

```
# Number of levels of each (categorical) covariate
NAge <- length(table(survey$age))
NEdu <- length(table(survey$edu))
```

Population loading

```
# population_eu: four-dimensional array containing population counts by
# nuts, gender, age and education group.
# The population data were obtained from Eurostat:
#
# → https://ec.europa.eu/eurostat/databrowser/view/cens_21cobe_r2/default/table?lang=en&category=cens.cens_21.cens
countries <- c("austria", "belgium", "switzerland", "germany", "spain",
               "france", "italy", "netherlands", "portugal")

population_list <- lapply(countries,
                          function(country) {
                            readRDS(file.path("../data", paste0("population-", country,
                                                                    "-eu.rds"))))
population_eu <- do.call(abind, c(population_list, along = 1))

# Checking
match(dimnames(population_eu)[[1]], cartography$NUTS_ID) - 1:NNUTS
```

```
## [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [38] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [75] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

```
# Gender selection
population_eu <- population_eu[, Gender, , ]
```

Ordinal model

```
### Model code ###

modelCode <- nimbleCode(
{
  # Likelihood
  for (Resp in 1:NResp) {
    y[Resp] ~ dcat(prlevels[Resp, 1:NCats])

    # Definition of the probabilities of each category as a function of the
    # cumulative probabilities
    prlevels[Resp, 1] <- p.gamma[Resp, 1]
    for (Cat in 2:(NCats-1)) {
      prlevels[Resp, Cat] <- p.gamma[Resp, Cat] - p.gamma[Resp, Cat-1]
    }
    prlevels[Resp, NCats] <- 1 - p.gamma[Resp, NCats-1]

    # Linear predictor
    for (Cat in 1:(NCats-1)) {
```

```

    logit(p.gamma[Resp, Cat]) <- kappa[Cat] -
      beta_age[age[Resp]] - beta_edu[edu[Resp]] -
      sd.theta * theta[nuts[Resp]]
  }
}

# Prior distributions

# kappa[1:(NCats-1)] cut points
# Monotonic transformation
for (Cat in 1:(NCats-1)) {
  kappa[Cat] <- logit(sum(delta[1:Cat]))
}
# delta[1:NCats] Dirichlet prior
delta[1:NCats] ~ ddirch(ones[1:NCats])

# beta_age[1:NAge] age group fixed effect (corner constraint)
beta_age[1] <- 0
for (AgeGroup in 2:NAge) {
  beta_age[AgeGroup] ~ dflat()
}

# beta_edu[1:NEdu] education group fixed effect (corner constraint)
beta_edu[1] <- 0
for (EduGroup in 2:NEdu) {
  beta_edu[EduGroup] ~ dflat()
}

# theta[1:NNUTS] spatial random effect
# LCAR distribution
theta[1:NNUTS] ~ dcar_leroux(rho = rho,
                             sd.theta = 1,
                             Lambda = Lambda[1:NNUTS],
                             from.to = from.to[1:NDist, 1:2])

# Hyperparameters of the spatial random effect
rho ~ dunif(0, 1)
sd.theta ~ dhalfflat()

# Stochastic restrictions in order to avoid confounding problems
# Required vectors
for (Resp in 1:NResp) {
  theta.Resp[Resp] <- theta[nuts[Resp]]
}

# Weighted constraint
# Zero-mean constraint for theta.Resp[1:NResp]
zero.theta.resp ~ dnorm(mean.thetas.resp, 10000)
mean.thetas.resp <- mean(theta.Resp[1:NResp])
}
)

### Data to be loaded ###

modelData <- list(y = y,
                  zero.theta.resp = 0)

```

```

modelConstants <- list(age = age, edu = edu, nuts = nuts,
                      NResp = NResp, NCats = NCats, NAge = NAge,
                      NEdu = NEdu, ones = ones, NNUTS = NNUTS,
                      NDist = NDist, Lambda = Lambda, from.to = from.to)

### Parameters to be saved ###

modelParameters <- c("kappa", "beta_age", "beta_edu",
                    "theta", "sd.theta", "rho")

### Loading functions for calling Nimble ###

source(
  ↪ "https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNimble.0.2.R")
load.leroux()

# Inital values
modelInits <- local({
  constants <- modelConstants
  function() {
    library(extraDistr)
    NAge <- constants$NAge
    NEdu <- constants$NEdu; NNUTS <- constants$NNUTS
    NCats <- constants$NCats; ones <- constants$ones
    list(delta = as.numeric(rdirichlet(1, ones)),
         beta_age = c(NA, rnorm(NAge - 1)),
         beta_edu = c(NA, rnorm(NEdu - 1)),
         rho = runif(1), sd.theta = runif(1),
         theta = rnorm(NNUTS, sd = 0.1))
  }
})

# # Number of chains to run in parallel
# nchains <- 5
# # pNimble call
# salnimble <- pNimble(code = modelCode, data = modelData, constants = modelConstants,
#                      inits = modelInits, nchains = nchains, seeds = 1:nchains,
#                      niter = 2000, nburnin = 1000, thin = 5,
#                      summary = TRUE, WAIC = TRUE, monitors = modelParameters,
#                      # ntifyAccount = "MigueBeneito",
#                      HMC = TRUE, parallel = TRUE)
#
# saveRDS(salnimble, file = file.path("../", "results", paste0("ordinal-leroux-hmc-waic-", gender,
#                      ↪ ".rds"))))

```

Bernoulli (D+) model

```

# Option 1 (D+): 0 = First two categories; 1 = Last three categories.
y[y > 5] <- NA
y[y == 1 | y == 2] <- 0
y[y == 3 | y == 4 | y == 5] <- 1

### Model code ###

modelCode <- nimbleCode(

```

```

{
  # Likelihood
  for (Resp in 1:NResp) {
    y[Resp] ~ dbern(prsuccess[Resp])

    # Linear predictor
    logit(prsuccess[Resp]) <- beta_0 +
      beta_age[age[Resp]] + beta_edu[edu[Resp]] +
      sd.theta * theta[nuts[Resp]]
  }

  # Prior distributions

  # beta_0 intercept
  beta_0 ~ dflat()

  # beta_age[1:NAge] age group fixed effect (corner constraint)
  beta_age[1] <- 0
  for (AgeGroup in 2:NAge) {
    beta_age[AgeGroup] ~ dflat()
  }

  # beta_edu[1:NEdu] education group fixed effect (corner constraint)
  beta_edu[1] <- 0
  for (EduGroup in 2:NEdu) {
    beta_edu[EduGroup] ~ dflat()
  }

  # theta[1:NNUTS] spatial random effect
  # LCAR distribution
  theta[1:NNUTS] ~ dcar_leroux(rho = rho,
                               sd.theta = 1,
                               Lambda = Lambda[1:NNUTS],
                               from.to = from.to[1:NDist, 1:2])

  # Hyperparameters of the spatial random effect
  rho ~ dunif(0, 1)
  sd.theta ~ dhalfflat()

  # Stochastic restrictions in order to avoid confounding problems
  # Required vectors
  for (Resp in 1:NResp) {
    theta.Resp[Resp] <- theta[nuts[Resp]]
  }

  # Weighted constraint
  # Zero-mean constraint for theta.Resp[1:NResp]
  zero.theta.resp ~ dnorm(mean.thetas.resp, 10000)
  mean.thetas.resp <- mean(theta.Resp[1:NResp])
}
)

### Data to be loaded ###

modelData <- list(y = y,
                  zero.theta.resp = 0)

```

```

modelConstants <- list(age = age, edu = edu, nuts = nuts,
                      NResp = NResp, NAge = NAge,
                      NEdu = NEdu, NNUTS = NNUTS,
                      NDist = NDist, Lambda = Lambda, from.to = from.to)

### Parameters to be saved ###

modelParameters <- c("beta_0", "beta_age", "beta_edu",
                    "theta", "sd.theta", "rho")

### Loading functions for calling Nimble ###

source(
  ↪ "https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNimble.0.2.R")
load.leroux()

# Inital values
modelInits <- local({
  constants <- modelConstants
  function() {
    NAge <- constants$NAge
    NEdu <- constants$NEdu; NNUTS <- constants$NNUTS
    list(beta_0 = rnorm(1),
         beta_age = c(NA, rnorm(NAge - 1)),
         beta_edu = c(NA, rnorm(NEdu - 1)),
         rho = runif(1), sd.theta = runif(1),
         theta = rnorm(NNUTS, sd = 0.1))
  }
})

# # Number of chains to run in parallel
# nchains <- 5
# # pNimble call
# salnimble <- pNimble(code = modelCode, data = modelData, constants = modelConstants,
#                      inits = modelInits, nchains = nchains, seeds = 1:nchains,
#                      niter = 2000, nburnin = 1000, thin = 5,
#                      summary = TRUE, WAIC = TRUE, monitors = modelParameters,
#                      ntifyAccount = "MigueBeneito",
#                      HMC = TRUE, parallel = TRUE)
#
# saveRDS(salnimble, file = file.path(".", "results", paste0("bernoulli-leroux-hmc-waic-",
#                      ↪ gender, "-o1.rds")))

```

Bernoulli (D-) model

```

# Option 2 (D-): 0 = First three categories; 1 = Last two categories.
y[y > 5] <- NA
y[y == 1 | y == 2 | y == 3] <- 0
y[y == 4 | y == 5] <- 1

### Model code ###

modelCode <- nimbleCode(
  {
    # Likelihood

```

```

for (Resp in 1:NResp) {
  y[Resp] ~ dbern(prsuccess[Resp])

  # Linear predictor
  logit(prsuccess[Resp]) <- beta_0 +
    beta_age[age[Resp]] + beta_edu[edu[Resp]] +
    sd.theta * theta[nuts[Resp]]
}

# Prior distributions

# beta_0 intercept
beta_0 ~ dflat()

# beta_age[1:NAge] age group fixed effect (corner constraint)
beta_age[1] <- 0
for (AgeGroup in 2:NAge) {
  beta_age[AgeGroup] ~ dflat()
}

# beta_edu[1:NEdu] education group fixed effect (corner constraint)
beta_edu[1] <- 0
for (EduGroup in 2:NEdu) {
  beta_edu[EduGroup] ~ dflat()
}

# theta[1:NNUTS] spatial random effect
# LCAR distribution
theta[1:NNUTS] ~ dcar_leroux(rho = rho,
                             sd.theta = 1,
                             Lambda = Lambda[1:NNUTS],
                             from.to = from.to[1:NDist, 1:2])

# Hyperparameters of the spatial random effect
rho ~ dunif(0, 1)
sd.theta ~ dhalfflat()

# Stochastic restrictions in order to avoid confounding problems
# Required vectors
for (Resp in 1:NResp) {
  theta.Resp[Resp] <- theta[nuts[Resp]]
}

# Weighted constraint
# Zero-mean constraint for theta.Resp[1:NResp]
zero.theta.resp ~ dnorm(mean.thetas.resp, 10000)
mean.thetas.resp <- mean(theta.Resp[1:NResp])
}
)

### Data to be loaded ###

modelData <- list(y = y,
                  zero.theta.resp = 0)

modelConstants <- list(age = age, edu = edu, nuts = nuts,
                       NResp = NResp, NAge = NAge,

```



```

      NEdu = NEdu, NNUTS = NNUTS,
      NDist = NDist, Lambda = Lambda, from.to = from.to)

### Parameters to be saved ###

modelParameters <- c("beta_0", "beta_age", "beta_edu",
                     "theta", "sd.theta", "rho")

### Loading functions for calling Nimble ###

source(
  ↪ "https://raw.githubusercontent.com/MigueBeneito/pNimble/refs/heads/main/RutinasNimble.0.2.R")
load.leroux()

# Inital values
modelInits <- local({
  constants <- modelConstants
  function() {
    NAge <- constants$NAge
    NEdu <- constants$NEdu; NNUTS <- constants$NNUTS
    list(beta_0 = rnorm(1),
          beta_age = c(NA, rnorm(NAge - 1)),
          beta_edu = c(NA, rnorm(NEdu - 1)),
          rho = runif(1), sd.theta = runif(1),
          theta = rnorm(NNUTS, sd = 0.1))
  }
})

# # Number of chains to run in parallel
# nchains <- 5
# # pNimble call
# salnimble <- pNimble(code = modelCode, data = modelData, constants = modelConstants,
#                       inits = modelInits, nchains = nchains, seeds = 1:nchains,
#                       niter = 2000, nburnin = 1000, thin = 5,
#                       summary = TRUE, WAIC = TRUE, monitors = modelParameters,
#                       # ntifyAccount = "MigueBeneito",
#                       HMC = TRUE, parallel = TRUE)
#
# saveRDS(salnimble, file = file.path("../", "results", paste0("bernoulli-leroux-hmc-waic-",
# ↪ gender, "-o2.rds"))))

```

Loading posterior samples

```

# Ordinal regression
ordinal_results <- readRDS(file = file.path("../", "results", paste0("ordinal-leroux-hmc-waic-",
  ↪ gender, ".rds")))
# Logistic regression (Option 1): 0 = First two categories; 1 = Last three categories.
bernoulli_results1 <- readRDS(file = file.path("../", "results",
  ↪ paste0("bernoulli-leroux-hmc-waic-", gender, "-o1.rds")))
# Logistic regression (Option 2): 0 = First three categories; 1 = Last two categories.
bernoulli_results2 <- readRDS(file = file.path("../", "results",
  ↪ paste0("bernoulli-leroux-hmc-waic-", gender, "-o2.rds")))

```

Convert NIMBLE output to WinBUGS-style format

```
nchains <- 5
nsims <- nchains * nrow(ordinal_results$samples[[1]])

# NimToWin:
# - transforms NIMBLE output to WinBUGS sims.list output format

NimToWin <- function(salnimble) {

  kappa <- matrix(nrow = nsims, ncol = NCats - 1)
  beta_age <- matrix(nrow = nsims, ncol = NAge)
  beta_edu <- matrix(nrow = nsims, ncol = NEdu)
  theta <- matrix(nrow = nsims, ncol = NNUTS)
  sd.theta <- numeric(length = nsims)
  rho <- numeric(length = nsims)

  for (Cat in 1:(NCats - 1)) {
    kappa[, Cat] <- c(salnimble[[1]][, paste0("kappa[", Cat, "]")],
                     salnimble[[2]][, paste0("kappa[", Cat, "]")],
                     salnimble[[3]][, paste0("kappa[", Cat, "]")],
                     salnimble[[4]][, paste0("kappa[", Cat, "]")],
                     salnimble[[5]][, paste0("kappa[", Cat, "]")])
  }

  for (AgeGroup in 1:NAge) {
    beta_age[, AgeGroup] <- c(salnimble[[1]][, paste0("beta_age[", AgeGroup, "]")],
                             salnimble[[2]][, paste0("beta_age[", AgeGroup, "]")],
                             salnimble[[3]][, paste0("beta_age[", AgeGroup, "]")],
                             salnimble[[4]][, paste0("beta_age[", AgeGroup, "]")],
                             salnimble[[5]][, paste0("beta_age[", AgeGroup, "]")])
  }

  for (EduGroup in 1:NEdu) {
    beta_edu[, EduGroup] <- c(salnimble[[1]][, paste0("beta_edu[", EduGroup, "]")],
                             salnimble[[2]][, paste0("beta_edu[", EduGroup, "]")],
                             salnimble[[3]][, paste0("beta_edu[", EduGroup, "]")],
                             salnimble[[4]][, paste0("beta_edu[", EduGroup, "]")],
                             salnimble[[5]][, paste0("beta_edu[", EduGroup, "]")])
  }

  for (NUTS in 1:NNUTS) {
    theta[, NUTS] <- c(salnimble[[1]][, paste0("theta[", NUTS, "]")],
                      salnimble[[2]][, paste0("theta[", NUTS, "]")],
                      salnimble[[3]][, paste0("theta[", NUTS, "]")],
                      salnimble[[4]][, paste0("theta[", NUTS, "]")],
                      salnimble[[5]][, paste0("theta[", NUTS, "]")])
  }

  sd.theta <- c(salnimble[[1]][, "sd.theta"], salnimble[[2]][, "sd.theta"],
                salnimble[[3]][, "sd.theta"], salnimble[[4]][, "sd.theta"],
                salnimble[[5]][, "sd.theta"])

  rho <- c(salnimble[[1]][, "rho"], salnimble[[2]][, "rho"],
           salnimble[[3]][, "rho"], salnimble[[4]][, "rho"],
           salnimble[[5]][, "rho"])

  summary <- MCMCsummary(object = salnimble, round = 4)
```

```

sims.list <- list("kappa" = kappa, "beta_age" = beta_age, "beta_edu" = beta_edu,
                 "theta" = theta, "sd.theta" = sd.theta, "rho" = rho)

salwinbugs <- list("summary" = summary, "sims.list" = sims.list,
                  "nchains" = nchains, "nsims" = nsims)

return(salwinbugs)
}

ordinal_salwinbugs <- NimToWin(salnimble = ordinal_results$samples)

NimToWin <- function(salnimble) {

  beta_0 <- numeric(length = nsims)
  beta_age <- matrix(nrow = nsims, ncol = NAge)
  beta_edu <- matrix(nrow = nsims, ncol = NEdu)
  theta <- matrix(nrow = nsims, ncol = NNUTS)
  sd.theta <- numeric(length = nsims)
  rho <- numeric(length = nsims)

  beta_0 <- c(salnimble[[1]][, "beta_0"], salnimble[[2]][, "beta_0"],
             salnimble[[3]][, "beta_0"], salnimble[[4]][, "beta_0"],
             salnimble[[5]][, "beta_0"])

  for (AgeGroup in 1:NAge) {
    beta_age[, AgeGroup] <- c(salnimble[[1]][, paste0("beta_age", AgeGroup, "")],
                             salnimble[[2]][, paste0("beta_age", AgeGroup, "")],
                             salnimble[[3]][, paste0("beta_age", AgeGroup, "")],
                             salnimble[[4]][, paste0("beta_age", AgeGroup, "")],
                             salnimble[[5]][, paste0("beta_age", AgeGroup, "")])
  }

  for (EduGroup in 1:NEdu) {
    beta_edu[, EduGroup] <- c(salnimble[[1]][, paste0("beta_edu", EduGroup, "")],
                             salnimble[[2]][, paste0("beta_edu", EduGroup, "")],
                             salnimble[[3]][, paste0("beta_edu", EduGroup, "")],
                             salnimble[[4]][, paste0("beta_edu", EduGroup, "")],
                             salnimble[[5]][, paste0("beta_edu", EduGroup, "")])
  }

  for (NUTS in 1:NNUTS) {
    theta[, NUTS] <- c(salnimble[[1]][, paste0("theta", NUTS, "")],
                      salnimble[[2]][, paste0("theta", NUTS, "")],
                      salnimble[[3]][, paste0("theta", NUTS, "")],
                      salnimble[[4]][, paste0("theta", NUTS, "")],
                      salnimble[[5]][, paste0("theta", NUTS, "")])
  }

  sd.theta <- c(salnimble[[1]][, "sd.theta"], salnimble[[2]][, "sd.theta"],
               salnimble[[3]][, "sd.theta"], salnimble[[4]][, "sd.theta"],
               salnimble[[5]][, "sd.theta"])

  rho <- c(salnimble[[1]][, "rho"], salnimble[[2]][, "rho"],
          salnimble[[3]][, "rho"], salnimble[[4]][, "rho"],
          salnimble[[5]][, "rho"])

  summary <- MCMCsummary(object = salnimble, round = 4)
  sims.list <- list("beta_0" = beta_0, "beta_age" = beta_age, "beta_edu" = beta_edu,

```

```

      "theta" = theta, "sd.theta" = sd.theta, "rho" = rho)

salwinbugs <- list("summary" = summary, "sims.list" = sims.list,
                 "nchains" = nchains, "nsims" = nsims)

return(salwinbugs)
}

bernoulli1_salwinbugs <- NimToWin(salnimble = bernoulli_results1$samples)
bernoulli2_salwinbugs <- NimToWin(salnimble = bernoulli_results2$samples)

```

Covariate effects

```

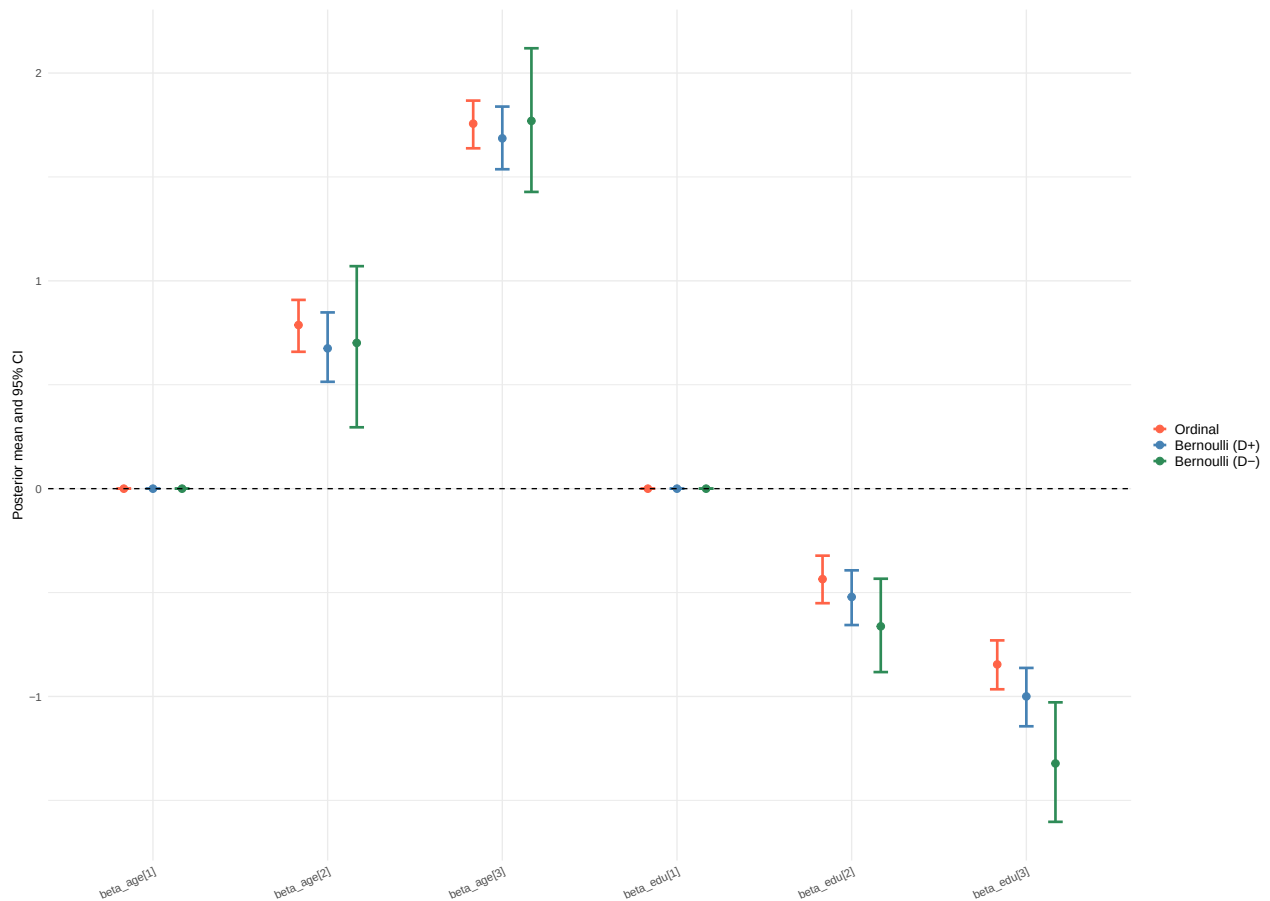
df_plot_ord <- data.frame("param" = rownames(ordinal_results$summary)[1:6],
                        "mean" = ordinal_results$summary[1:6, 1],
                        "lower" = ordinal_results$summary[1:6, 3],
                        "upper" = ordinal_results$summary[1:6, 5],
                        "model" = "Ordinal")
df_plot_bern1 <- data.frame("param" = rownames(bernoulli_results1$summary)[2:7],
                          "mean" = bernoulli_results1$summary[2:7, 1],
                          "lower" = bernoulli_results1$summary[2:7, 3],
                          "upper" = bernoulli_results1$summary[2:7, 5],
                          "model" = "Bernoulli (D+)")
df_plot_bern2 <- data.frame("param" = rownames(bernoulli_results2$summary)[2:7],
                          "mean" = bernoulli_results2$summary[2:7, 1],
                          "lower" = bernoulli_results2$summary[2:7, 3],
                          "upper" = bernoulli_results2$summary[2:7, 5],
                          "model" = "Bernoulli (D-)")
df_plot <- rbind(df_plot_ord, df_plot_bern1, df_plot_bern2)
df_plot$param <- factor(df_plot$param, levels = df_plot$param[1:6])
df_plot$model <- factor(df_plot$model, levels = c("Ordinal", "Bernoulli (D+)", "Bernoulli (D-)"))

p_fixed <- ggplot(df_plot, aes(x = param, y = mean, color = model)) +
  geom_point(position = position_dodge(width = 0.5), size = 2.5) +
  geom_errorbar(aes(ymin = lower, ymax = upper),
               position = position_dodge(width = 0.5), width = 0.25, linewidth = 1) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  scale_color_manual(values = c("Ordinal" = "tomato",
                                "Bernoulli (D+)" = "steelblue",
                                "Bernoulli (D-)" = "seagreen")) +

  theme_minimal() +
  theme(axis.text.x = element_text(angle = 25, hjust = 1),
        legend.text = element_text(size = 11),
        legend.key.size = unit(0.45, "cm")) +
  labs(x = NULL, y = "Posterior mean and 95% CI", color = "")

p_fixed

```



Spatial effect

```
# Posterior mean of theta's
cartography$thetamean_ordinal <-
→ ordinal_results$summary[startsWith(rownames(ordinal_results$summary), "theta"), 1]
cartography$thetamean_bernoulli1 <-
→ bernoulli_results1$summary[startsWith(rownames(bernoulli_results1$summary), "theta"), 1]
cartography$thetamean_bernoulli2 <-
→ bernoulli_results2$summary[startsWith(rownames(bernoulli_results2$summary), "theta"), 1]

# 95% credible intervals of theta's
cartography$thetalow_ordinal <-
→ ordinal_results$summary[startsWith(rownames(ordinal_results$summary), "theta"), 3]
cartography$thetap_ordinal <-
→ ordinal_results$summary[startsWith(rownames(ordinal_results$summary), "theta"), 5]
cartography$thetalow_bernoulli1 <-
→ bernoulli_results1$summary[startsWith(rownames(bernoulli_results1$summary), "theta"), 3]
cartography$thetap_bernoulli1 <-
→ bernoulli_results1$summary[startsWith(rownames(bernoulli_results1$summary), "theta"), 5]
cartography$thetalow_bernoulli2 <-
→ bernoulli_results2$summary[startsWith(rownames(bernoulli_results2$summary), "theta"), 3]
cartography$thetap_bernoulli2 <-
→ bernoulli_results2$summary[startsWith(rownames(bernoulli_results2$summary), "theta"), 5]

# Posterior sd of theta's
```

```

cartography$thetasd_ordinal <-
  ↳ ordinal_results$summary[startsWith(rownames(ordinal_results$summary), "theta"), 2]
cartography$thetasd_bernoulli1 <-
  ↳ bernoulli_results1$summary[startsWith(rownames(bernoulli_results1$summary), "theta"), 2]
cartography$thetasd_bernoulli2 <-
  ↳ bernoulli_results2$summary[startsWith(rownames(bernoulli_results2$summary), "theta"), 2]

# Checking when theta is greater than zero
ordinal_stepsim <- 1 * (ordinal_salwinbugs$sims.list$theta > 0)
bernoulli1_stepsim <- 1 * (bernoulli1_salwinbugs$sims.list$theta > 0)
bernoulli2_stepsim <- 1 * (bernoulli2_salwinbugs$sims.list$theta > 0)

# Posterior probabilities of theta's
cartography$probmean_ordinal <- apply(ordinal_stepsim, 2, mean)
cartography$probmean_bernoulli1 <- apply(bernoulli1_stepsim, 2, mean)
cartography$probmean_bernoulli2 <- apply(bernoulli2_stepsim, 2, mean)

# 95% credible intervals of posterior probabilities
cartography$problow_ordinal <- apply(ordinal_stepsim, 2, quantile, probs = 0.025)
cartography$probup_ordinal <- apply(ordinal_stepsim, 2, quantile, probs = 0.975)
cartography$problow_bernoulli1 <- apply(bernoulli1_stepsim, 2, quantile, probs = 0.025)
cartography$probup_bernoulli1 <- apply(bernoulli1_stepsim, 2, quantile, probs = 0.975)
cartography$problow_bernoulli2 <- apply(bernoulli2_stepsim, 2, quantile, probs = 0.025)
cartography$probup_bernoulli2 <- apply(bernoulli2_stepsim, 2, quantile, probs = 0.975)

# Borders for mean and sd rows: theta interval does not include zero
cartography$border_mean_ordinal <- NA
cartography$border_mean_ordinal[cartography$thetalow_ordinal > 0] <- "#543005"
cartography$border_mean_ordinal[cartography$thetaup_ordinal < 0] <- "#1B7837"

cartography$border_mean_bernoulli1 <- NA
cartography$border_mean_bernoulli1[cartography$thetalow_bernoulli1 > 0] <- "#543005"
cartography$border_mean_bernoulli1[cartography$thetaup_bernoulli1 < 0] <- "#1B7837"

cartography$border_mean_bernoulli2 <- NA
cartography$border_mean_bernoulli2[cartography$thetalow_bernoulli2 > 0] <- "#543005"
cartography$border_mean_bernoulli2[cartography$thetaup_bernoulli2 < 0] <- "#1B7837"

cartography$border_sd_ordinal <- NA
cartography$border_sd_ordinal[cartography$thetalow_ordinal > 0 | cartography$thetaup_ordinal < 0]
  ↳ <- "#08306B"

cartography$border_sd_bernoulli1 <- NA
cartography$border_sd_bernoulli1[cartography$thetalow_bernoulli1 > 0 |
  ↳ cartography$thetaup_bernoulli1 < 0] <- "#08306B"

cartography$border_sd_bernoulli2 <- NA
cartography$border_sd_bernoulli2[cartography$thetalow_bernoulli2 > 0 |
  ↳ cartography$thetaup_bernoulli2 < 0] <- "#08306B"

# Borders for significance row: probability interval does not include 0.5
cartography$border_prob_ordinal <- NA
cartography$border_prob_ordinal[cartography$problow_ordinal > 0.5] <- "#67000D"
cartography$border_prob_ordinal[cartography$probup_ordinal < 0.5] <- "#00441B"

cartography$border_prob_bernoulli1 <- NA
cartography$border_prob_bernoulli1[cartography$problow_bernoulli1 > 0.5] <- "#67000D"
cartography$border_prob_bernoulli1[cartography$probup_bernoulli1 < 0.5] <- "#00441B"

```

```

cartography$border_prob_bernoulli2 <- NA
cartography$border_prob_bernoulli2[cartography$problow_bernoulli2 > 0.5] <- "#67000D"
cartography$border_prob_bernoulli2[cartography$probup_bernoulli2 < 0.5] <- "#00441B"

# Mean
limit <- max(abs(c(cartography$thetamean_ordinal,
                    cartography$thetamean_bernoulli1,
                    cartography$thetamean_bernoulli2)), na.rm = TRUE)

p_thetamean_ordinal <- ggplot(cartography) +
  geom_sf(aes(fill = thetamean_ordinal, color = border_mean_ordinal), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "BrBG")[9:1],
                      limits = c(-limit, limit),
                      values = rescale(c(-limit, 0, limit)),
                      name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_thetamean_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = thetamean_bernoulli1, color = border_mean_bernoulli1), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "BrBG")[9:1],
                      limits = c(-limit, limit),
                      values = rescale(c(-limit, 0, limit)),
                      name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_thetamean_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = thetamean_bernoulli2, color = border_mean_bernoulli2), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "BrBG")[9:1],
                      limits = c(-limit, limit),
                      values = rescale(c(-limit, 0, limit)),
                      name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

# Sd
limit <- c(min(c(cartography$thetasd_ordinal,
                  cartography$thetasd_bernoulli1,
                  cartography$thetasd_bernoulli2), na.rm = TRUE),
           max(c(cartography$thetasd_ordinal,
                  cartography$thetasd_bernoulli1,
                  cartography$thetasd_bernoulli2), na.rm = TRUE))

p_thetasd_ordinal <- ggplot(cartography) +
  geom_sf(aes(fill = thetasd_ordinal, color = border_sd_ordinal), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
                      limits = c(limit[1], limit[2]),
                      name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_thetasd_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = thetasd_bernoulli1, color = border_sd_bernoulli1), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
                      limits = c(limit[1], limit[2]),
                      name = NULL) +

```

```

scale_color_identity(na.value = NA) +
theme_void()

p_thetasd_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = thetasd_bernoulli2, color = border_sd_bernoulli2), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
    limits = c(limit[1], limit[2]),
    name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

# Significance
p_probmean_ordinal <- ggplot(cartography) +
  geom_sf(aes(fill = probmean_ordinal, color = border_prob_ordinal), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlGn")[9:1],
    limits = c(0, 1),
    name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_probmean_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = probmean_bernoulli1, color = border_prob_bernoulli1), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlGn")[9:1],
    limits = c(0, 1),
    name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_probmean_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = probmean_bernoulli2, color = border_prob_bernoulli2), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlGn")[9:1],
    limits = c(0, 1),
    name = NULL) +
  scale_color_identity(na.value = NA) +
  theme_void()

p_thetamean_ordinal <- p_thetamean_ordinal + ggtitle("Ordinal")
p_thetamean_bernoulli1 <- p_thetamean_bernoulli1 + ggtitle("Bernoulli (D+)")
p_thetamean_bernoulli2 <- p_thetamean_bernoulli2 + ggtitle("Bernoulli (D-)")

p_thetamean_ordinal <- p_thetamean_ordinal + labs(tag = "Mean")
p_thetasd_ordinal <- p_thetasd_ordinal + labs(tag = "Sd")
p_probmean_ordinal <- p_probmean_ordinal + labs(tag = "P(> 0)")

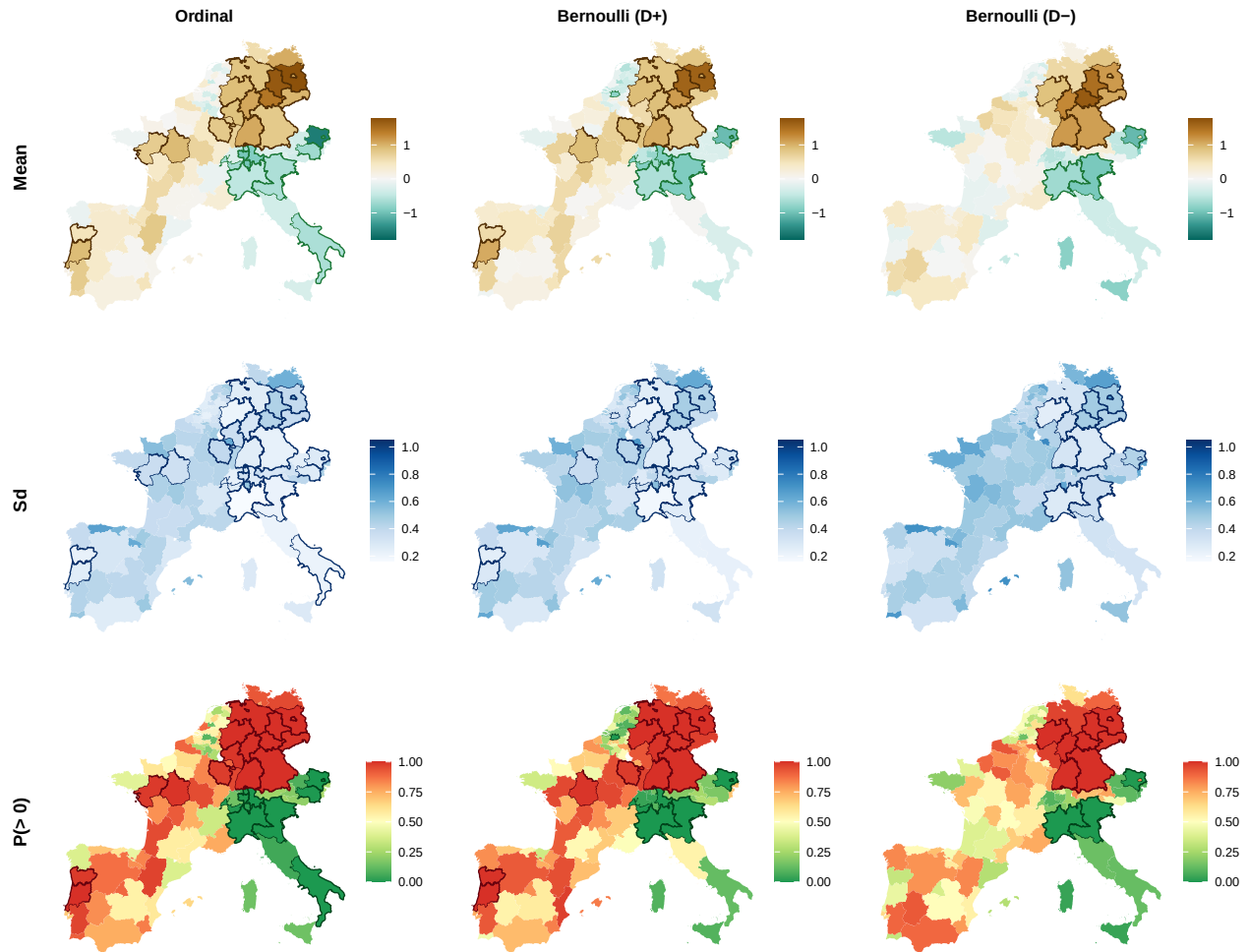
tema_mapas <- theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
  plot.tag = element_text(face = "bold", size = 13, angle = 90),
  plot.tag.position = c(-0.08, 0.5),
  plot.margin = margin(5.5, 5.5, 5.5, 20))

final_plot <- wrap_plots(p_thetamean_ordinal + tema_mapas,
  p_thetamean_bernoulli1 + tema_mapas,
  p_thetamean_bernoulli2 + tema_mapas,
  p_thetasd_ordinal + tema_mapas,
  p_thetasd_bernoulli1 + tema_mapas,
  p_thetasd_bernoulli2 + tema_mapas,
  p_probmean_ordinal + tema_mapas,
  p_probmean_bernoulli1 + tema_mapas,
  p_probmean_bernoulli2 + tema_mapas, ncol = 3)

```



```
final_plot
```



Poststratified estimates

```
# poststratify:
# - (1) computes the nsims simulated probabilities for each age, education and nuts
# - (2) post-stratifies these probabilities for each nuts and category

poststratify <- function(salwinbugs) {

  p.gamma <- array(dim = c(nsims, NNUTS, NAge, NEdu, NCats - 1))
  prlevels <- array(dim = c(nsims, NNUTS, NAge, NEdu, NCats))
  prlevels_post <- array(dim = c(nsims, NNUTS, NCats))
  population <- population_eu

  # Probabilities for each age, education and nuts
  for (sim in 1:nsims) {
    for (AgeGroup in 1:NAge) {
      for (EduGroup in 1:NEdu) {
        for (NUTS in 1:NNUTS) {
          for (Cat in 1:(NCats - 1)) {
```

```

        p.gamma[sim, NUTS, AgeGroup, EduGroup, Cat] <-
            ilogit(salwinbugs$sims.list$kappa[sim, Cat] -
                  salwinbugs$sims.list$beta_age[sim, AgeGroup] -
                  salwinbugs$sims.list$beta_edu[sim, EduGroup] -
                  salwinbugs$sims.list$sd.theta[sim] * salwinbugs$sims.list$theta[sim,
→ NUTS]))
    }

    prlevels[sim, NUTS, AgeGroup, EduGroup, 1] <- p.gamma[sim, NUTS, AgeGroup, EduGroup, 1]
    prlevels[sim, NUTS, AgeGroup, EduGroup, NCats] <- 1 - p.gamma[sim, NUTS, AgeGroup,
→ EduGroup, NCats - 1]

    for (Cat in 2:(NCats - 1)) {
        prlevels[sim, NUTS, AgeGroup, EduGroup, Cat] <-
            p.gamma[sim, NUTS, AgeGroup, EduGroup, Cat] - p.gamma[sim, NUTS, AgeGroup,
→ EduGroup, Cat-1]
    }
}
}
}

# Post-stratification
for (NUTS in 1:NNUTS) {
    for (Cat in 1:NCats) {
        prlevels_post[sim, NUTS, Cat] <- sum(population[NUTS, , ])^(-1) * sum(prlevels[sim, NUTS,
→ , , Cat] * population[NUTS, , ])
    }
}
if (sim %in% c(1, seq(nsims/nchains, nsims, nsims/nchains))) {
    cat(sim, "of", nsims, "simulations", "\n")
} else {}
}

return(prlevels_post)
}

ordinal_post <- poststratify(salwinbugs = ordinal_salwinbugs)

```

```

## 1 of 1000 simulations
## 200 of 1000 simulations
## 400 of 1000 simulations
## 600 of 1000 simulations
## 800 of 1000 simulations
## 1000 of 1000 simulations

```

```

poststratify <- function(salwinbugs) {

    prsuccess <- array(dim = c(nsims, NNUTS, NAge, NEdu))
    prsuccess_post <- matrix(nrow = nsims, ncol = NNUTS)
    population <- population_eu

    # Probabilities for each age, education and nuts
    for (sim in 1:nsims) {
        for (AgeGroup in 1:NAge) {
            for (EduGroup in 1:NEdu) {
                for (NUTS in 1:NNUTS) {
                    prsuccess[sim, NUTS, AgeGroup, EduGroup] <-

```

```

        ilogit(salwinbugs$sims.list$beta_0[sim] +
              salwinbugs$sims.list$beta_age[sim, AgeGroup] +
              salwinbugs$sims.list$beta_edu[sim, EduGroup] +
              salwinbugs$sims.list$sd.theta[sim] * salwinbugs$sims.list$theta[sim, NUTS])
      }
    }
  }

  # Post-stratification
  for (NUTS in 1:NNUTS) {
    prsuccess_post[sim, NUTS] <- sum(population[NUTS, , ])^(-1) * sum(prsuccess[sim, NUTS, , ]
→ * population[NUTS, , ])
  }
  if (sim %in% c(1, seq(nsims/nchains, nsims, nsims/nchains))) {
    cat(sim, "of", nsims, "simulations", "\n")
  } else {}
}

return(prsuccess_post)
}

bernoulli1_post <- poststratify(salwinbugs = bernoulli1_salwinbugs)

```

```

## 1 of 1000 simulations
## 200 of 1000 simulations
## 400 of 1000 simulations
## 600 of 1000 simulations
## 800 of 1000 simulations
## 1000 of 1000 simulations

```

```
bernoulli2_post <- poststratify(salwinbugs = bernoulli2_salwinbugs)
```

```

## 1 of 1000 simulations
## 200 of 1000 simulations
## 400 of 1000 simulations
## 600 of 1000 simulations
## 800 of 1000 simulations
## 1000 of 1000 simulations

```

```

# Post-stratified posterior means and sd's of the percentages for each nuts and category

### Ordinal model ###

# Mean
cartography$percentage_mean1 <- apply(ordinal_post, 2:3, mean)[, 1] * 100
p_percentage_mean1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean1, color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  theme_void()
cartography$percentage_mean2 <- apply(ordinal_post, 2:3, mean)[, 2] * 100
p_percentage_mean2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean2, color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  theme_void()
cartography$percentage_mean3 <- apply(ordinal_post, 2:3, mean)[, 3] * 100
p_percentage_mean3 <- ggplot(cartography) +

```

```

    geom_sf(aes(fill = percentage_mean3), color = "grey30", linewidth = 0.1) +
    scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
    theme_void()
cartography$percentage_mean4 <- apply(ordinal_post, 2:3, mean)[, 4] * 100
p_percentage_mean4 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean4), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  theme_void()
cartography$percentage_mean5 <- apply(ordinal_post, 2:3, mean)[, 5] * 100
p_percentage_mean5 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean5), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  theme_void()

# Sd
cartography$percentage_sd1 <- apply(ordinal_post, 2:3, sd)[, 1] * 100
p_percentage_sd1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd1), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()
cartography$percentage_sd2 <- apply(ordinal_post, 2:3, sd)[, 2] * 100
p_percentage_sd2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()
cartography$percentage_sd3 <- apply(ordinal_post, 2:3, sd)[, 3] * 100
p_percentage_sd3 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd3), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()
cartography$percentage_sd4 <- apply(ordinal_post, 2:3, sd)[, 4] * 100
p_percentage_sd4 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd4), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()
cartography$percentage_sd5 <- apply(ordinal_post, 2:3, sd)[, 5] * 100
p_percentage_sd5 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd5), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()

# CV
cartography$percentage_CV1 <- 100 * cartography$percentage_sd1/cartography$percentage_mean1
p_percentage_CV1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV1), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()
cartography$percentage_CV2 <- 100 * cartography$percentage_sd2/cartography$percentage_mean2
p_percentage_CV2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()
cartography$percentage_CV3 <- 100 * cartography$percentage_sd3/cartography$percentage_mean3
p_percentage_CV3 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV3), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()
cartography$percentage_CV4 <- 100 * cartography$percentage_sd4/cartography$percentage_mean4

```

```

p_percentage_CV4 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV4), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()
cartography$percentage_CV5 <- 100 * cartography$percentage_sd5/cartography$percentage_mean5
p_percentage_CV5 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV5), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()

# Probability above European mean
eu_mean1 <- apply(ordinal_post[, , 1], 1, mean)
eu_mean2 <- apply(ordinal_post[, , 2], 1, mean)
eu_mean3 <- apply(ordinal_post[, , 3], 1, mean)
eu_mean4 <- apply(ordinal_post[, , 4], 1, mean)
eu_mean5 <- apply(ordinal_post[, , 5], 1, mean)

eu_mean_post1 <- mean(eu_mean1) * 100
eu_mean_post2 <- mean(eu_mean2) * 100
eu_mean_post3 <- mean(eu_mean3) * 100
eu_mean_post4 <- mean(eu_mean4) * 100
eu_mean_post5 <- mean(eu_mean5) * 100

cartography$percentage_prob_above1 <- apply(ordinal_post[, , 1] > eu_mean1, 2, mean)
cartography$percentage_prob_above2 <- apply(ordinal_post[, , 2] > eu_mean2, 2, mean)
cartography$percentage_prob_above3 <- apply(ordinal_post[, , 3] > eu_mean3, 2, mean)
cartography$percentage_prob_above4 <- apply(ordinal_post[, , 4] > eu_mean4, 2, mean)
cartography$percentage_prob_above5 <- apply(ordinal_post[, , 5] > eu_mean5, 2, mean)

cartography$percentage_low1 <- apply(ordinal_post[, , 1] * 100, 2, quantile, probs = 0.025)
cartography$percentage_low2 <- apply(ordinal_post[, , 2] * 100, 2, quantile, probs = 0.025)
cartography$percentage_low3 <- apply(ordinal_post[, , 3] * 100, 2, quantile, probs = 0.025)
cartography$percentage_low4 <- apply(ordinal_post[, , 4] * 100, 2, quantile, probs = 0.025)
cartography$percentage_low5 <- apply(ordinal_post[, , 5] * 100, 2, quantile, probs = 0.025)

cartography$percentage_up1 <- apply(ordinal_post[, , 1] * 100, 2, quantile, probs = 0.975)
cartography$percentage_up2 <- apply(ordinal_post[, , 2] * 100, 2, quantile, probs = 0.975)
cartography$percentage_up3 <- apply(ordinal_post[, , 3] * 100, 2, quantile, probs = 0.975)
cartography$percentage_up4 <- apply(ordinal_post[, , 4] * 100, 2, quantile, probs = 0.975)
cartography$percentage_up5 <- apply(ordinal_post[, , 5] * 100, 2, quantile, probs = 0.975)

cartography$border_prob_above1 <- NA
cartography$border_prob_above1[cartography$percentage_low1 > eu_mean_post1] <- "#67000D"
cartography$border_prob_above1[cartography$percentage_up1 < eu_mean_post1] <- "#08306B"

cartography$border_prob_above2 <- NA
cartography$border_prob_above2[cartography$percentage_low2 > eu_mean_post2] <- "#67000D"
cartography$border_prob_above2[cartography$percentage_up2 < eu_mean_post2] <- "#08306B"

cartography$border_prob_above3 <- NA
cartography$border_prob_above3[cartography$percentage_low3 > eu_mean_post3] <- "#67000D"
cartography$border_prob_above3[cartography$percentage_up3 < eu_mean_post3] <- "#08306B"

cartography$border_prob_above4 <- NA
cartography$border_prob_above4[cartography$percentage_low4 > eu_mean_post4] <- "#67000D"
cartography$border_prob_above4[cartography$percentage_up4 < eu_mean_post4] <- "#08306B"

cartography$border_prob_above5 <- NA

```

```

cartography$border_prob_above5[cartography$percentage_low5 > eu_mean_post5] <- "#67000D"
cartography$border_prob_above5[cartography$percentage_up5 < eu_mean_post5] <- "#08306B"

p_percentage_prob_above1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above1, color = border_prob_above1), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_prob_above2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above2, color = border_prob_above2), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_prob_above3 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above3, color = border_prob_above3), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_prob_above4 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above4, color = border_prob_above4), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_prob_above5 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above5, color = border_prob_above5), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_mean1 <- p_percentage_mean1 + ggtitle(y_labels[1])
p_percentage_mean2 <- p_percentage_mean2 + ggtitle(y_labels[2])
p_percentage_mean3 <- p_percentage_mean3 + ggtitle(y_labels[3])
p_percentage_mean4 <- p_percentage_mean4 + ggtitle(y_labels[4])
p_percentage_mean5 <- p_percentage_mean5 + ggtitle(y_labels[5])

p_percentage_mean1 <- p_percentage_mean1 + labs(tag = "Mean")
p_percentage_sd1 <- p_percentage_sd1 + labs(tag = "Sd")
p_percentage_CV1 <- p_percentage_CV1 + labs(tag = "CV (%)")
p_percentage_prob_above1 <- p_percentage_prob_above1 + labs(tag = "P(> EU mean)")

tema_mapas <- theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
  plot.tag = element_text(face = "bold", size = 12, angle = 90),
  plot.tag.position = c(-0.08, 0.5),
  plot.margin = margin(5.5, 5.5, 5.5, 20))

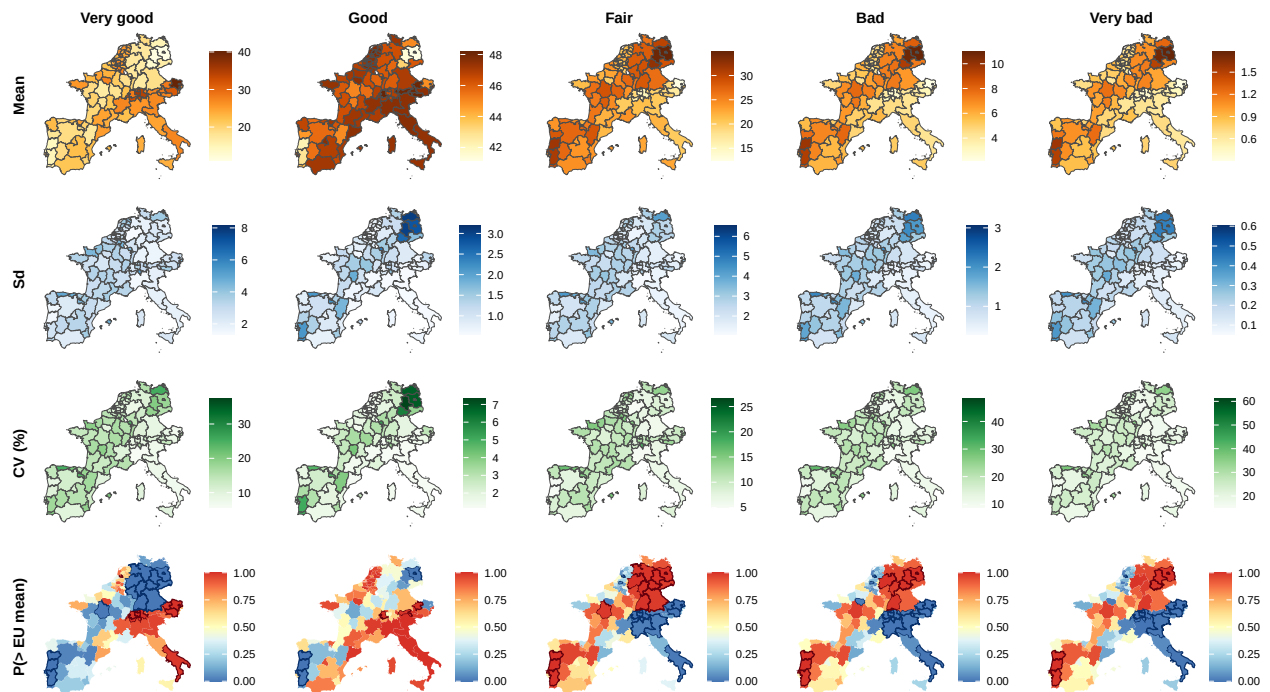
final_plot <- wrap_plots(p_percentage_mean1 + tema_mapas,
  p_percentage_mean2 + tema_mapas,
  p_percentage_mean3 + tema_mapas,
  p_percentage_mean4 + tema_mapas,
  p_percentage_mean5 + tema_mapas,
  p_percentage_sd1 + tema_mapas,
  p_percentage_sd2 + tema_mapas,
  p_percentage_sd3 + tema_mapas,
  p_percentage_sd4 + tema_mapas,

```

```

p_percentage_sd5 + tema_mapas,
p_percentage_CV1 + tema_mapas,
p_percentage_CV2 + tema_mapas,
p_percentage_CV3 + tema_mapas,
p_percentage_CV4 + tema_mapas,
p_percentage_CV5 + tema_mapas,
p_percentage_prob_above1 + tema_mapas,
p_percentage_prob_above2 + tema_mapas,
p_percentage_prob_above3 + tema_mapas,
p_percentage_prob_above4 + tema_mapas,
p_percentage_prob_above5 + tema_mapas, ncol = 5)
final_plot

```



```

### Bernoulli models: D+ and D- ###

tema_mapas_bern <- theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
  plot.tag = element_text(face = "bold", size = 12, angle = 90),
  plot.tag.position = c(-0.08, 0.5),
  plot.margin = margin(5.5, 5.5, 5.5, 20))

### Bernoulli (D+) ###

cartography$percentage_mean_Dplus <- apply(bernoulli1_post, 2, mean) * 100
p_percentage_mean_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  ggtitle("Mean") + theme_void()

cartography$percentage_sd_Dplus <- apply(bernoulli1_post, 2, sd) * 100
p_percentage_sd_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  ggtitle("Sd") + theme_void()

```



```

cartography$percentage_CV_Dplus <- 100 * cartography$percentage_sd_Dplus /
  ↪ cartography$percentage_mean_Dplus
p_percentage_CV_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  ggtitle("CV (%)") + theme_void()

eu_mean_Dplus <- apply(bernoulli1_post, 1, mean)
eu_mean_post_Dplus <- mean(eu_mean_Dplus) * 100

cartography$percentage_prob_above_Dplus <- apply(bernoulli1_post > eu_mean_Dplus, 2, mean)
cartography$percentage_low_Dplus <- apply(bernoulli1_post * 100, 2, quantile, probs = 0.025)
cartography$percentage_up_Dplus <- apply(bernoulli1_post * 100, 2, quantile, probs = 0.975)

cartography$border_prob_above_Dplus <- NA
cartography$border_prob_above_Dplus[cartography$percentage_low_Dplus > eu_mean_post_Dplus] <-
  ↪ "#67000D"
cartography$border_prob_above_Dplus[cartography$percentage_up_Dplus < eu_mean_post_Dplus] <-
  ↪ "#08306B"

p_percentage_prob_above_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above_Dplus, color = border_prob_above_Dplus), linewidth =
  ↪ 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + ggtitle("P(> EU mean)") + theme_void()

p_percentage_mean_Dplus <- p_percentage_mean_Dplus + labs(tag = "Bernoulli (D+)")

### Bernoulli (D-) ###

cartography$percentage_mean_Dminus <- apply(bernoulli2_post, 2, mean) * 100
p_percentage_mean_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"), name = NULL) +
  theme_void()

cartography$percentage_sd_Dminus <- apply(bernoulli2_post, 2, sd) * 100
p_percentage_sd_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), name = NULL) +
  theme_void()

cartography$percentage_CV_Dminus <- 100 * cartography$percentage_sd_Dminus /
  ↪ cartography$percentage_mean_Dminus
p_percentage_CV_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"), name = NULL) +
  theme_void()

eu_mean_Dminus <- apply(bernoulli2_post, 1, mean)
eu_mean_post_Dminus <- mean(eu_mean_Dminus) * 100

cartography$percentage_prob_above_Dminus <- apply(bernoulli2_post > eu_mean_Dminus, 2, mean)
cartography$percentage_low_Dminus <- apply(bernoulli2_post * 100, 2, quantile, probs = 0.025)
cartography$percentage_up_Dminus <- apply(bernoulli2_post * 100, 2, quantile, probs = 0.975)

```



```

cartography$border_prob_above_Dminus <- NA
cartography$border_prob_above_Dminus[cartography$percentage_low_Dminus > eu_mean_post_Dminus] <-
  ↪ "#67000D"
cartography$border_prob_above_Dminus[cartography$percentage_up_Dminus < eu_mean_post_Dminus] <-
  ↪ "#08306B"

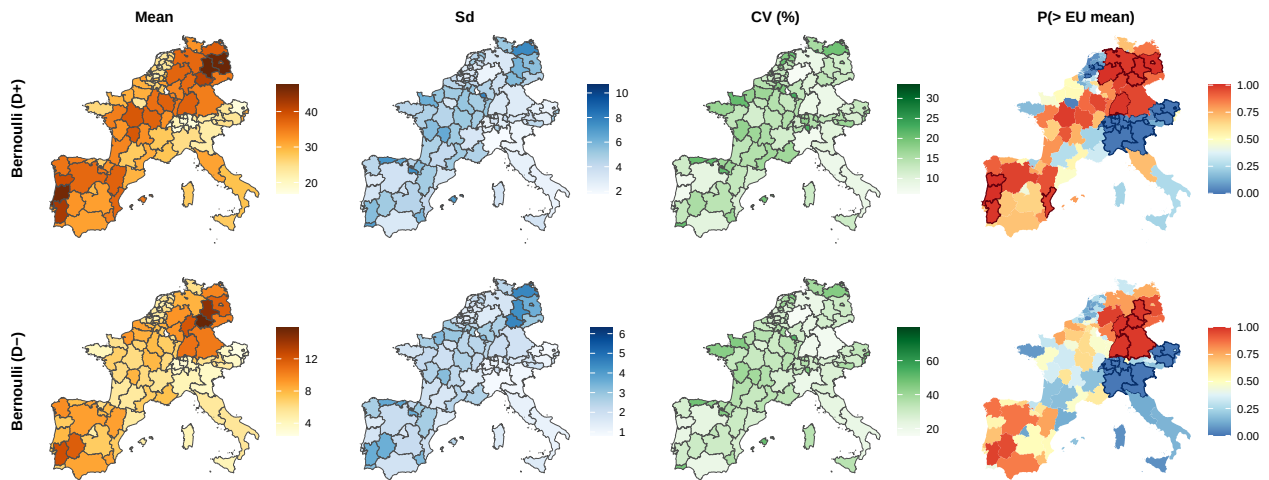
p_percentage_prob_above_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_prob_above_Dminus, color = border_prob_above_Dminus), linewidth =
    ↪ 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_percentage_mean_Dminus <- p_percentage_mean_Dminus + labs(tag = "Bernoulli (D-)")

final_plot_bernoulli <- wrap_plots(p_percentage_mean_Dplus + tema_mapas_bern,
  p_percentage_sd_Dplus + tema_mapas_bern,
  p_percentage_CV_Dplus + tema_mapas_bern,
  p_percentage_prob_above_Dplus + tema_mapas_bern,
  p_percentage_mean_Dminus + tema_mapas_bern,
  p_percentage_sd_Dminus + tema_mapas_bern,
  p_percentage_CV_Dminus + tema_mapas_bern,
  p_percentage_prob_above_Dminus + tema_mapas_bern,
  ncol = 4)

final_plot_bernoulli

```



Comparison: Bernoulli and aggregated ordinal models

```

# Ordinal aggregations
ordinal_Dplus_post <- ordinal_post[, , 3] + ordinal_post[, , 4] + ordinal_post[, , 5]
ordinal_Dminus_post <- ordinal_post[, , 4] + ordinal_post[, , 5]

# Mean
cartography$percentage_mean_bernoulli1 <- apply(bernoulli1_post, 2, mean) * 100
cartography$percentage_mean_ordinal_Dplus <- apply(ordinal_Dplus_post, 2, mean) * 100
cartography$percentage_mean_bernoulli2 <- apply(bernoulli2_post, 2, mean) * 100
cartography$percentage_mean_ordinal_Dminus <- apply(ordinal_Dminus_post, 2, mean) * 100

```

```

# Sd
cartography$percentage_sd_bernoulli1 <- apply(bernoulli1_post, 2, sd) * 100
cartography$percentage_sd_ordinal_Dplus <- apply(ordinal_Dplus_post, 2, sd) * 100
cartography$percentage_sd_bernoulli2 <- apply(bernoulli2_post, 2, sd) * 100
cartography$percentage_sd_ordinal_Dminus <- apply(ordinal_Dminus_post, 2, sd) * 100

# CV
cartography$percentage_CV_bernoulli1 <- 100 * cartography$percentage_sd_bernoulli1 /
  ↪ cartography$percentage_mean_bernoulli1
cartography$percentage_CV_ordinal_Dplus <- 100 * cartography$percentage_sd_ordinal_Dplus /
  ↪ cartography$percentage_mean_ordinal_Dplus
cartography$percentage_CV_bernoulli2 <- 100 * cartography$percentage_sd_bernoulli2 /
  ↪ cartography$percentage_mean_bernoulli2
cartography$percentage_CV_ordinal_Dminus <- 100 * cartography$percentage_sd_ordinal_Dminus /
  ↪ cartography$percentage_mean_ordinal_Dminus

# Common scales within each comparison
limit_mean_Dplus <- range(c(cartography$percentage_mean_bernoulli1,
  cartography$percentage_mean_ordinal_Dplus), na.rm = TRUE)
limit_mean_Dminus <- range(c(cartography$percentage_mean_bernoulli2,
  cartography$percentage_mean_ordinal_Dminus), na.rm = TRUE)

limit_sd_Dplus <- range(c(cartography$percentage_sd_bernoulli1,
  cartography$percentage_sd_ordinal_Dplus), na.rm = TRUE)
limit_sd_Dminus <- range(c(cartography$percentage_sd_bernoulli2,
  cartography$percentage_sd_ordinal_Dminus), na.rm = TRUE)

limit_CV_Dplus <- range(c(cartography$percentage_CV_bernoulli1,
  cartography$percentage_CV_ordinal_Dplus), na.rm = TRUE)
limit_CV_Dminus <- range(c(cartography$percentage_CV_bernoulli2,
  cartography$percentage_CV_ordinal_Dminus), na.rm = TRUE)

# Mean maps
p_percentage_mean_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_bernoulli1), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"),
    limits = limit_mean_Dplus,
    name = NULL) + ggtitle("Bernoulli (D+)") + theme_void()

p_percentage_mean_ordinal_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_ordinal_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"),
    limits = limit_mean_Dplus,
    name = NULL) + ggtitle("Ordinal (D+)") + theme_void()

p_percentage_mean_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_bernoulli2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"),
    limits = limit_mean_Dminus,
    name = NULL) + ggtitle("Bernoulli (D-)") + theme_void()

p_percentage_mean_ordinal_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_mean_ordinal_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "YlOrBr"),
    limits = limit_mean_Dminus,
    name = NULL) + ggtitle("Ordinal (D-)") + theme_void()

# Sd maps

```

```

p_percentage_sd_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_bernoulli1), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
    limits = limit_sd_Dplus,
    name = NULL) + theme_void()

p_percentage_sd_ordinal_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_ordinal_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
    limits = limit_sd_Dplus,
    name = NULL) + theme_void()

p_percentage_sd_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_bernoulli2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
    limits = limit_sd_Dminus,
    name = NULL) + theme_void()

p_percentage_sd_ordinal_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_sd_ordinal_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"),
    limits = limit_sd_Dminus,
    name = NULL) + theme_void()

# CV maps
p_percentage_CV_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_bernoulli1), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"),
    limits = limit_CV_Dplus,
    name = NULL) + theme_void()

p_percentage_CV_ordinal_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_ordinal_Dplus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"),
    limits = limit_CV_Dplus,
    name = NULL) + theme_void()

p_percentage_CV_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_bernoulli2), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"),
    limits = limit_CV_Dminus,
    name = NULL) + theme_void()

p_percentage_CV_ordinal_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = percentage_CV_ordinal_Dminus), color = "grey30", linewidth = 0.1) +
  scale_fill_gradientn(colours = brewer.pal(9, "Greens"),
    limits = limit_CV_Dminus,
    name = NULL) + theme_void()

# Probability above European mean

eu_mean_bernoulli1 <- apply(bernoulli1_post, 1, mean)
eu_mean_ordinal_Dplus <- apply(ordinal_Dplus_post, 1, mean)
eu_mean_bernoulli2 <- apply(bernoulli2_post, 1, mean)
eu_mean_ordinal_Dminus <- apply(ordinal_Dminus_post, 1, mean)

eu_mean_post_bernoulli1 <- mean(eu_mean_bernoulli1) * 100
eu_mean_post_ordinal_Dplus <- mean(eu_mean_ordinal_Dplus) * 100

```

```

eu_mean_post_bernoulli2 <- mean(eu_mean_bernoulli2) * 100
eu_mean_post_ordinal_Dminus <- mean(eu_mean_ordinal_Dminus) * 100

cartography$prob_above_bernoulli1 <- apply(bernoulli1_post > eu_mean_bernoulli1, 2, mean)
cartography$prob_above_ordinal_Dplus <- apply(ordinal_Dplus_post > eu_mean_ordinal_Dplus, 2,
  ↪ mean)
cartography$prob_above_bernoulli2 <- apply(bernoulli2_post > eu_mean_bernoulli2, 2, mean)
cartography$prob_above_ordinal_Dminus <- apply(ordinal_Dminus_post > eu_mean_ordinal_Dminus, 2,
  ↪ mean)

cartography$low_bernoulli1 <- apply(bernoulli1_post * 100, 2, quantile, probs = 0.025)
cartography$up_bernoulli1 <- apply(bernoulli1_post * 100, 2, quantile, probs = 0.975)

cartography$low_ordinal_Dplus <- apply(ordinal_Dplus_post * 100, 2, quantile, probs = 0.025)
cartography$up_ordinal_Dplus <- apply(ordinal_Dplus_post * 100, 2, quantile, probs = 0.975)

cartography$low_bernoulli2 <- apply(bernoulli2_post * 100, 2, quantile, probs = 0.025)
cartography$up_bernoulli2 <- apply(bernoulli2_post * 100, 2, quantile, probs = 0.975)

cartography$low_ordinal_Dminus <- apply(ordinal_Dminus_post * 100, 2, quantile, probs = 0.025)
cartography$up_ordinal_Dminus <- apply(ordinal_Dminus_post * 100, 2, quantile, probs = 0.975)

cartography$border_above_bernoulli1 <- NA
cartography$border_above_bernoulli1[cartography$low_bernoulli1 > eu_mean_post_bernoulli1] <-
  ↪ "#67000D"
cartography$border_above_bernoulli1[cartography$up_bernoulli1 < eu_mean_post_bernoulli1] <-
  ↪ "#08306B"

cartography$border_above_ordinal_Dplus <- NA
cartography$border_above_ordinal_Dplus[cartography$low_ordinal_Dplus >
  ↪ eu_mean_post_ordinal_Dplus] <- "#67000D"
cartography$border_above_ordinal_Dplus[cartography$up_ordinal_Dplus < eu_mean_post_ordinal_Dplus]
  ↪ <- "#08306B"

cartography$border_above_bernoulli2 <- NA
cartography$border_above_bernoulli2[cartography$low_bernoulli2 > eu_mean_post_bernoulli2] <-
  ↪ "#67000D"
cartography$border_above_bernoulli2[cartography$up_bernoulli2 < eu_mean_post_bernoulli2] <-
  ↪ "#08306B"

cartography$border_above_ordinal_Dminus <- NA
cartography$border_above_ordinal_Dminus[cartography$low_ordinal_Dminus >
  ↪ eu_mean_post_ordinal_Dminus] <- "#67000D"
cartography$border_above_ordinal_Dminus[cartography$up_ordinal_Dminus <
  ↪ eu_mean_post_ordinal_Dminus] <- "#08306B"

p_prob_above_bernoulli1 <- ggplot(cartography) +
  geom_sf(aes(fill = prob_above_bernoulli1, color = border_above_bernoulli1), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_prob_above_ordinal_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = prob_above_ordinal_Dplus, color = border_above_ordinal_Dplus), linewidth =
  ↪ 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

```

```

p_prob_above_bernoulli2 <- ggplot(cartography) +
  geom_sf(aes(fill = prob_above_bernoulli2, color = border_above_bernoulli2), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

p_prob_above_ordinal_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = prob_above_ordinal_Dminus, color = border_above_ordinal_Dminus), linewidth =
    ↪ 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

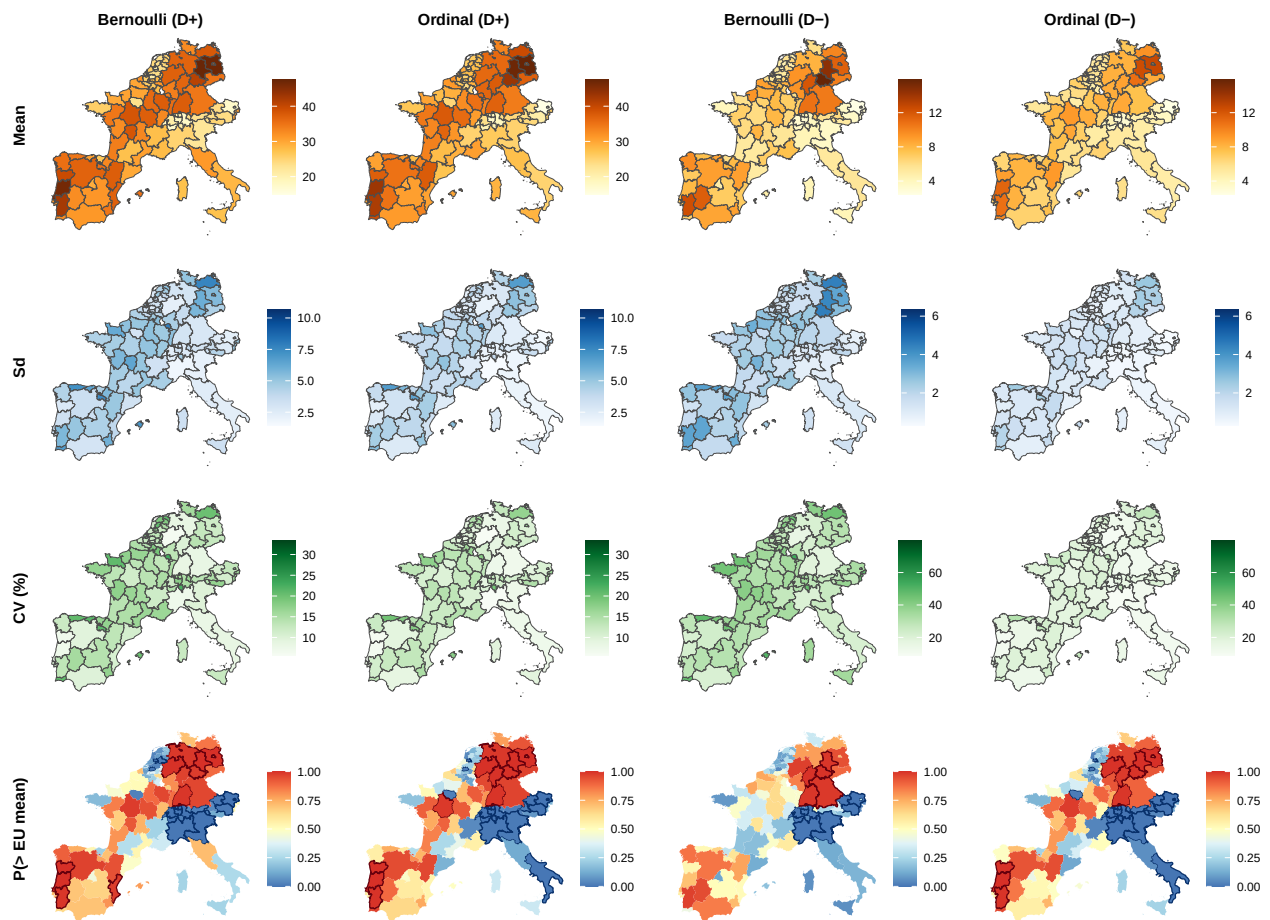
# Row labels
p_percentage_mean_bernoulli1 <- p_percentage_mean_bernoulli1 + labs(tag = "Mean")
p_percentage_sd_bernoulli1 <- p_percentage_sd_bernoulli1 + labs(tag = "Sd")
p_percentage_CV_bernoulli1 <- p_percentage_CV_bernoulli1 + labs(tag = "CV (%)")
p_prob_above_bernoulli1 <- p_prob_above_bernoulli1 + labs(tag = "P(> EU mean)")

# Theme
tema_mapas <- theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
  plot.tag = element_text(face = "bold", size = 12, angle = 90),
  plot.tag.position = c(-0.08, 0.5), plot.margin = margin(5.5, 5.5, 5.5, 20))

final_plot <- wrap_plots(p_percentage_mean_bernoulli1 + tema_mapas,
  p_percentage_mean_ordinal_Dplus + tema_mapas,
  p_percentage_mean_bernoulli2 + tema_mapas,
  p_percentage_mean_ordinal_Dminus + tema_mapas,
  p_percentage_sd_bernoulli1 + tema_mapas,
  p_percentage_sd_ordinal_Dplus + tema_mapas,
  p_percentage_sd_bernoulli2 + tema_mapas,
  p_percentage_sd_ordinal_Dminus + tema_mapas,
  p_percentage_CV_bernoulli1 + tema_mapas,
  p_percentage_CV_ordinal_Dplus + tema_mapas,
  p_percentage_CV_bernoulli2 + tema_mapas,
  p_percentage_CV_ordinal_Dminus + tema_mapas,
  p_prob_above_bernoulli1 + tema_mapas,
  p_prob_above_ordinal_Dplus + tema_mapas,
  p_prob_above_bernoulli2 + tema_mapas,
  p_prob_above_ordinal_Dminus + tema_mapas, ncol = 4)

final_plot

```



Differences: Bernoulli and aggregated ordinal models

```
ordinal_Dplus_post <- ordinal_post[, , 3] + ordinal_post[, , 4] + ordinal_post[, , 5]
ordinal_Dminus_post <- ordinal_post[, , 4] + ordinal_post[, , 5]

diff_Dplus_post <- ordinal_Dplus_post - bernoulli1_post
diff_Dminus_post <- ordinal_Dminus_post - bernoulli2_post

# Mean
cartography$diffmean_Dplus <- apply(diff_Dplus_post, 2, mean) * 100
cartography$diffmean_Dminus <- apply(diff_Dminus_post, 2, mean) * 100

# Sd
cartography$diffsd_Dplus <- apply(diff_Dplus_post, 2, sd) * 100
cartography$diffsd_Dminus <- apply(diff_Dminus_post, 2, sd) * 100

# 95% credible intervals for the differences
cartography$diffflow_Dplus <- apply(diff_Dplus_post * 100, 2, quantile, probs = 0.025)
cartography$diffup_Dplus <- apply(diff_Dplus_post * 100, 2, quantile, probs = 0.975)

cartography$diffflow_Dminus <- apply(diff_Dminus_post * 100, 2, quantile, probs = 0.025)
cartography$diffup_Dminus <- apply(diff_Dminus_post * 100, 2, quantile, probs = 0.975)

# Probability that the difference is greater than zero
```

```

Dplus_stepsim <- 1 * (diff_Dplus_post > 0)
Dminus_stepsim <- 1 * (diff_Dminus_post > 0)

cartography$prob_diff_Dplus_great0 <- apply(Dplus_stepsim, 2, mean)
cartography$prob_diff_Dminus_great0 <- apply(Dminus_stepsim, 2, mean)

cartography$problow_Dplus <- apply(Dplus_stepsim, 2, quantile, probs = 0.025)
cartography$probup_Dplus <- apply(Dplus_stepsim, 2, quantile, probs = 0.975)

cartography$problow_Dminus <- apply(Dminus_stepsim, 2, quantile, probs = 0.025)
cartography$probup_Dminus <- apply(Dminus_stepsim, 2, quantile, probs = 0.975)

# Borders
cartography$border_mean_Dplus <- NA
cartography$border_mean_Dplus[cartography$diffflow_Dplus > 0] <- "#67000D"
cartography$border_mean_Dplus[cartography$diffup_Dplus < 0] <- "#08306B"

cartography$border_mean_Dminus <- NA
cartography$border_mean_Dminus[cartography$diffflow_Dminus > 0] <- "#67000D"
cartography$border_mean_Dminus[cartography$diffup_Dminus < 0] <- "#08306B"

cartography$border_sd_Dplus <- NA
cartography$border_sd_Dplus[cartography$diffflow_Dplus > 0 | cartography$diffup_Dplus < 0] <-
  → "#08306B"

cartography$border_sd_Dminus <- NA
cartography$border_sd_Dminus[cartography$diffflow_Dminus > 0 | cartography$diffup_Dminus < 0] <-
  → "#08306B"

cartography$border_prob_Dplus <- NA
cartography$border_prob_Dplus[cartography$problow_Dplus > 0.5] <- "#67000D"
cartography$border_prob_Dplus[cartography$probup_Dplus < 0.5] <- "#08306B"

cartography$border_prob_Dminus <- NA
cartography$border_prob_Dminus[cartography$problow_Dminus > 0.5] <- "#67000D"
cartography$border_prob_Dminus[cartography$probup_Dminus < 0.5] <- "#08306B"

# Common scales
limit_diffmean <- max(abs(c(cartography$diffmean_Dplus,
                             cartography$diffmean_Dminus))), na.rm = TRUE)

limit_diffsd <- range(c(cartography$diffsd_Dplus,
                         cartography$diffsd_Dminus), na.rm = TRUE)

# Mean maps
p_diffmean_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = diffmean_Dplus, color = border_mean_Dplus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
                       limits = c(-limit_diffmean, limit_diffmean),
                       values = rescale(c(-limit_diffmean, 0, limit_diffmean)),
                       name = NULL) +
  scale_color_identity(na.value = NA) + ggtitle("Mean") + theme_void()

p_diffmean_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = diffmean_Dminus, color = border_mean_Dminus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
                       limits = c(-limit_diffmean, limit_diffmean),
                       values = rescale(c(-limit_diffmean, 0, limit_diffmean)),

```



```

        name = NULL) +
scale_color_identity(na.value = NA) + theme_void()

# Sd maps
p_diffsd_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = diffsd_Dplus, color = border_sd_Dplus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), limits = limit_diffsd, name = NULL) +
  scale_color_identity(na.value = NA) + ggtitle("Sd") + theme_void()

p_diffsd_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = diffsd_Dminus, color = border_sd_Dminus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "Blues"), limits = limit_diffsd, name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

# Probability maps
p_prob_Dplus <- ggplot(cartography) +
  geom_sf(aes(fill = prob_diff_Dplus_great0, color = border_prob_Dplus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + ggtitle("P(> 0)") + theme_void()

p_prob_Dminus <- ggplot(cartography) +
  geom_sf(aes(fill = prob_diff_Dminus_great0, color = border_prob_Dminus), linewidth = 0.45) +
  scale_fill_gradientn(colours = brewer.pal(9, "RdYlBu")[9:1],
    limits = c(0, 1), values = rescale(c(0, 0.5, 1)), name = NULL) +
  scale_color_identity(na.value = NA) + theme_void()

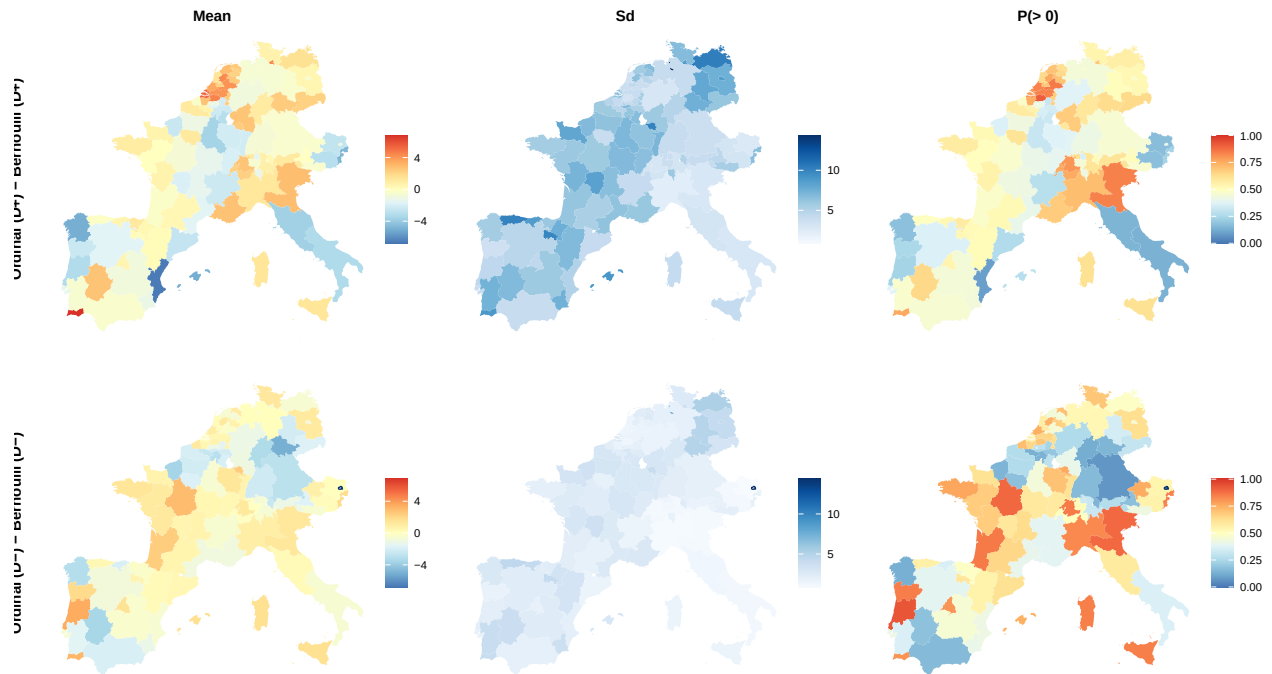
# Row labels
p_diffmean_Dplus <- p_diffmean_Dplus + labs(tag = "Ordinal (D+) - Bernoulli (D+)")
p_diffmean_Dminus <- p_diffmean_Dminus + labs(tag = "Ordinal (D-) - Bernoulli (D-)")

# Theme
tema_mapas_diff <- theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
  plot.tag = element_text(face = "bold", size = 12, angle = 90),
  plot.tag.position = c(-0.08, 0.5),
  plot.margin = margin(5.5, 5.5, 5.5, 20))

# Final plot
final_plot_diff <- wrap_plots(p_diffmean_Dplus + tema_mapas_diff,
  p_diffsd_Dplus + tema_mapas_diff,
  p_prob_Dplus + tema_mapas_diff,
  p_diffmean_Dminus + tema_mapas_diff,
  p_diffsd_Dminus + tema_mapas_diff,
  p_prob_Dminus + tema_mapas_diff, ncol = 3)

final_plot_diff

```

Ordinal profiles in selected regions

```
capital_nuts <- c("AT13", "BE10", "CH02", "DE3", "ES30",
                  "FR10", "IT1", "NL32", "PT17")

selected_ids <- match(capital_nuts, cartography$NUTS_ID)

df_capitals <- data.frame()
for (capital in selected_ids) {

  ordinal_area <- ordinal_post[, capital, ] * 100
  bernoulli1_area <- apply(bernoulli1_post, 2, mean)[capital] * 100
  bernoulli2_area <- apply(bernoulli2_post, 2, mean)[capital] * 100

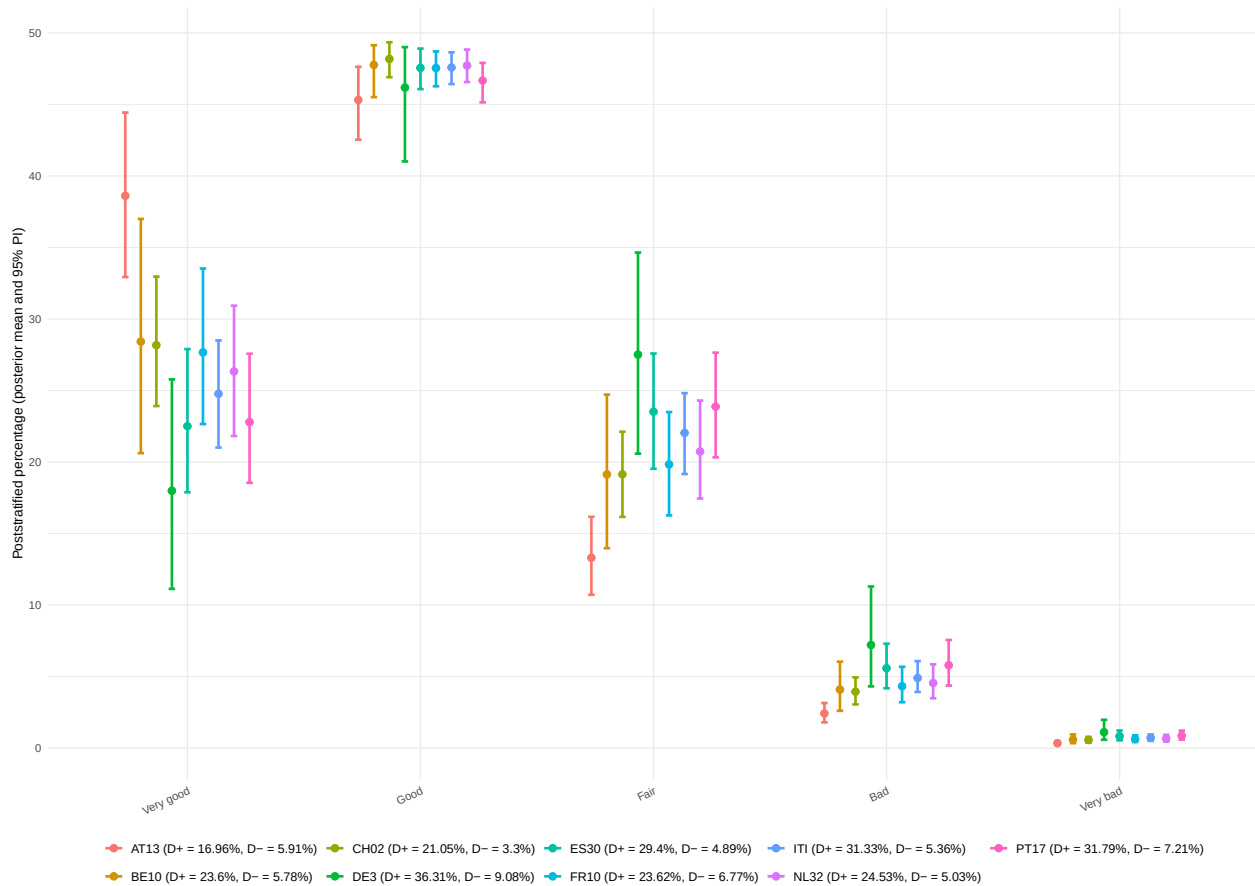
  df_capitals <- rbind(df_capitals, data.frame(
    category = c("Very good", "Good", "Fair", "Bad", "Very bad"),
    mean = apply(ordinal_area, 2, mean),
    lower = apply(ordinal_area, 2, quantile, probs = 0.025),
    upper = apply(ordinal_area, 2, quantile, probs = 0.975),
    area = paste0(cartography$NUTS_ID[capital],
                  " (D+ = ", round(bernoulli1_area, 2), "%",
                  ", D- = ", round(bernoulli2_area, 2), "%)"))
})

df_capitals$category <- factor(df_capitals$category,
                              levels = c("Very good", "Good", "Fair", "Bad", "Very bad"))

p_capitals <- ggplot(df_capitals, aes(x = category, y = mean, color = area)) +
  geom_point(position = position_dodge(width = 0.6), size = 2.5) +
  geom_errorbar(aes(ymin = lower, ymax = upper),
               position = position_dodge(width = 0.6), width = 0.25, linewidth = 1) +
  theme_minimal() + labs(x = NULL, y = "Poststratified percentage (posterior mean and 95% PI)",
    ↪ color = NULL) +
```

```
theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
      axis.text.x = element_text(angle = 25, hjust = 1),
      legend.position = "bottom", legend.text = element_text(size = 10.25),
      legend.key.size = unit(0.6, "cm"))
```

p_capitals



Convergence assessment

```
### Ordinal model ###
```

```
round(ordinal_results$summary, 4)
```

##		mean	sd	2.5%	50%	97.5%	Rhat	n.eff
##	beta_age[1]	0.0000	0.0000	0.0000	0.0000	0.0000	NaN	0
##	beta_age[2]	0.7874	0.0607	0.6586	0.7843	0.9083	1.01	1274
##	beta_age[3]	1.7564	0.0580	1.6378	1.7588	1.8673	1.00	1125
##	beta_edu[1]	0.0000	0.0000	0.0000	0.0000	0.0000	NaN	0
##	beta_edu[2]	-0.4354	0.0576	-0.5512	-0.4347	-0.3224	1.01	951
##	beta_edu[3]	-0.8457	0.0596	-0.9654	-0.8452	-0.7301	1.01	1000
##	delta[1]	0.3179	0.0130	0.2913	0.3185	0.3431	1.00	1044
##	delta[2]	0.5155	0.0072	0.5016	0.5155	0.5293	1.01	1000
##	delta[3]	0.1399	0.0074	0.1266	0.1396	0.1548	1.00	1058
##	delta[4]	0.0234	0.0018	0.0203	0.0233	0.0272	1.00	1000

## delta[5]	0.0032	0.0004	0.0024	0.0032	0.0042	1.00	1045
## kappa[1]	-0.7640	0.0600	-0.8892	-0.7609	-0.6493	1.00	1043
## kappa[2]	1.6116	0.0640	1.4837	1.6119	1.7287	1.00	1026
## kappa[3]	3.6010	0.0752	3.4452	3.6020	3.7405	1.00	1000
## kappa[4]	5.7455	0.1362	5.4713	5.7395	6.0179	1.00	1047
## rho	0.6339	0.1528	0.3316	0.6482	0.9019	1.00	1037
## sd.theta	0.5232	0.0619	0.4133	0.5191	0.6527	1.00	835
## theta[1]	-0.2871	0.4646	-1.1842	-0.2893	0.6331	1.00	1118
## theta[2]	-1.5527	0.2836	-2.1323	-1.5409	-1.0213	1.00	1000
## theta[3]	-1.1531	0.3074	-1.7505	-1.1476	-0.6109	1.01	894
## theta[4]	-0.3437	0.4124	-1.1815	-0.3508	0.4664	1.00	1102
## theta[5]	-0.6914	0.2768	-1.2643	-0.6896	-0.1729	1.00	972
## theta[6]	-0.3487	0.2522	-0.8498	-0.3405	0.1087	1.01	1006
## theta[7]	-0.3091	0.3412	-0.9618	-0.3215	0.3759	1.00	939
## theta[8]	-0.2206	0.3119	-0.8320	-0.2188	0.3956	1.00	916
## theta[9]	-0.3228	0.3875	-1.0584	-0.3308	0.4193	1.01	1076
## theta[10]	-0.1339	0.4390	-1.0816	-0.1407	0.7298	1.01	959
## theta[11]	-0.3025	0.2369	-0.7736	-0.3055	0.1631	1.00	1000
## theta[12]	-0.3802	0.3319	-1.0121	-0.3853	0.2607	1.00	1208
## theta[13]	0.0452	0.2931	-0.5340	0.0559	0.5903	1.00	1062
## theta[14]	0.0535	0.2944	-0.5133	0.0678	0.6088	1.00	1077
## theta[15]	0.2376	0.3245	-0.3603	0.2405	0.8928	1.00	899
## theta[16]	-0.2703	0.4796	-1.1940	-0.2558	0.5914	1.01	992
## theta[17]	-0.2173	0.3287	-0.9014	-0.1995	0.4147	1.00	1083
## theta[18]	0.1105	0.3497	-0.5840	0.0981	0.7580	1.00	1209
## theta[19]	-0.3010	0.4849	-1.2121	-0.2921	0.5716	1.00	943
## theta[20]	-0.4263	0.4615	-1.2998	-0.4257	0.4231	1.01	1000
## theta[21]	-0.6578	0.2735	-1.2308	-0.6446	-0.1400	1.00	1000
## theta[22]	-0.2473	0.2414	-0.7220	-0.2408	0.2031	1.00	1941
## theta[23]	-0.5191	0.3264	-1.1705	-0.5305	0.1028	1.01	1055
## theta[24]	-0.6911	0.3279	-1.3560	-0.6713	-0.0967	1.00	1070
## theta[25]	-0.8635	0.2831	-1.4321	-0.8525	-0.3097	1.00	1107
## theta[26]	-0.9342	0.3371	-1.5962	-0.9232	-0.2952	1.01	1000
## theta[27]	-0.9399	0.5068	-1.9967	-0.9294	-0.0163	1.00	944
## theta[28]	1.0466	0.2634	0.5401	1.0416	1.5980	1.00	1000
## theta[29]	0.7915	0.2378	0.3526	0.7826	1.2667	1.00	1063
## theta[30]	1.0649	0.5690	0.0483	1.0616	2.2266	1.01	939
## theta[31]	1.7645	0.3929	1.0234	1.7473	2.5741	1.00	1101
## theta[32]	0.4997	1.0061	-1.4241	0.4927	2.5292	1.01	1000
## theta[33]	0.4234	0.5922	-0.7398	0.4309	1.5327	1.00	1418
## theta[34]	1.0647	0.3099	0.4648	1.0613	1.6999	1.01	956
## theta[35]	1.0288	0.5989	-0.1733	1.0481	2.1989	1.00	969
## theta[36]	0.8466	0.2723	0.3115	0.8419	1.3899	1.00	1110
## theta[37]	0.9051	0.2193	0.5115	0.8946	1.3596	1.01	1000
## theta[38]	0.8759	0.3656	0.1190	0.8671	1.6255	1.01	1000
## theta[39]	0.7550	0.6220	-0.4886	0.7657	1.9260	1.00	1066
## theta[40]	0.9629	0.3875	0.2740	0.9547	1.7496	1.00	1063
## theta[41]	1.6854	0.4487	0.7773	1.6848	2.5664	1.01	1000
## theta[42]	0.5705	0.4087	-0.1599	0.5510	1.4116	1.01	1146
## theta[43]	1.4231	0.4380	0.5870	1.4212	2.3122	1.00	1101
## theta[44]	-0.1054	0.3863	-0.8829	-0.0932	0.6413	1.00	1140
## theta[45]	0.3296	0.6604	-0.9415	0.3320	1.5571	1.00	1388
## theta[46]	0.1306	0.5853	-1.0208	0.1652	1.2098	1.00	1000
## theta[47]	-0.0613	0.4135	-0.8496	-0.0665	0.7510	1.00	1057

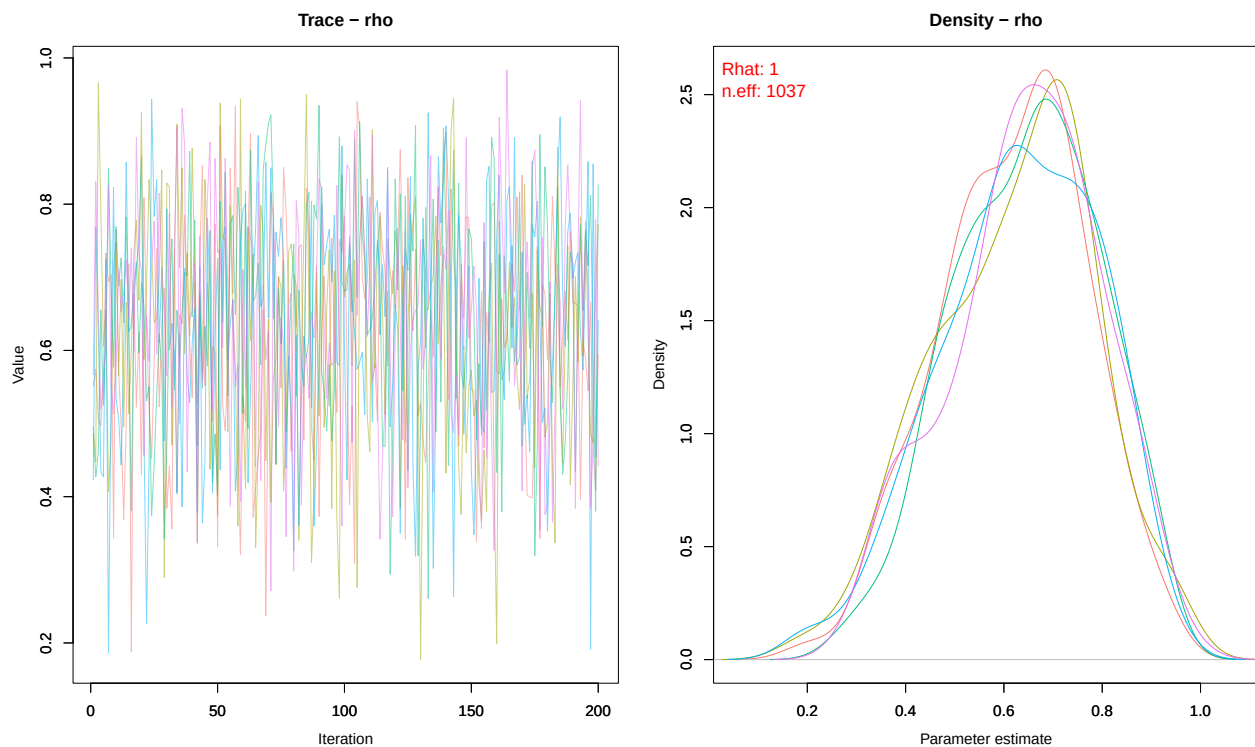
## theta[48]	0.4906	0.4925	-0.3807	0.4682	1.4391	1.01	971
## theta[49]	0.5284	0.6229	-0.7742	0.5130	1.7040	1.01	1000
## theta[50]	0.8031	0.4317	-0.0698	0.8106	1.6603	1.00	1049
## theta[51]	0.3787	0.3105	-0.2187	0.3791	0.9914	1.01	1046
## theta[52]	0.3561	0.3127	-0.2739	0.3593	0.9390	1.01	1090
## theta[53]	0.0347	0.4123	-0.7928	0.0522	0.8049	1.00	1045
## theta[54]	0.3754	0.4235	-0.4847	0.3813	1.2142	1.00	1004
## theta[55]	-0.0897	0.2906	-0.6660	-0.0870	0.4873	1.00	990
## theta[56]	0.0677	0.3361	-0.6198	0.0724	0.7029	1.00	955
## theta[57]	0.2063	0.5574	-0.8862	0.1888	1.2776	1.00	1045
## theta[58]	0.1818	0.2744	-0.3453	0.1666	0.7431	1.00	1052
## theta[59]	0.4854	0.5026	-0.5159	0.4828	1.4910	1.00	1052
## theta[60]	0.0636	0.2922	-0.5335	0.0603	0.6570	1.01	994
## theta[61]	0.9280	0.3520	0.2487	0.9444	1.5870	1.01	1097
## theta[62]	0.6787	0.4207	-0.1507	0.6771	1.5273	1.00	1000
## theta[63]	0.3119	0.4332	-0.5286	0.3094	1.1423	1.00	1373
## theta[64]	0.1174	0.5479	-0.9751	0.1286	1.2618	1.01	947
## theta[65]	0.0073	0.4784	-0.8891	0.0084	0.9762	1.00	1039
## theta[66]	0.5041	0.3995	-0.2447	0.5037	1.3265	1.00	1049
## theta[67]	0.1331	0.4218	-0.6960	0.1411	0.9196	1.00	1062
## theta[68]	0.7107	0.4345	-0.1433	0.7060	1.6196	1.00	852
## theta[69]	0.4382	0.4443	-0.4473	0.4593	1.3291	1.00	1005
## theta[70]	0.7963	0.3979	0.0287	0.8014	1.5744	1.00	1108
## theta[71]	0.7592	0.3717	0.0770	0.7345	1.4993	1.00	1008
## theta[72]	-0.0805	0.3954	-0.8087	-0.1033	0.7499	1.00	1682
## theta[73]	0.6103	0.3805	-0.0979	0.6109	1.3449	1.00	950
## theta[74]	0.6849	0.4891	-0.2905	0.6972	1.6355	1.00	1355
## theta[75]	0.3946	0.4530	-0.5087	0.3774	1.2427	1.00	1145
## theta[76]	0.0645	0.4347	-0.7968	0.0742	0.8664	1.00	1046
## theta[77]	0.0697	0.3743	-0.6424	0.0584	0.8361	1.00	923
## theta[78]	0.2895	0.4274	-0.5624	0.2902	1.0890	1.01	1680
## theta[79]	-0.1092	0.3001	-0.6880	-0.1047	0.4825	1.00	1118
## theta[80]	0.2776	0.4167	-0.5165	0.2861	1.0908	1.00	953
## theta[81]	-0.4912	0.1682	-0.8460	-0.4784	-0.1771	1.00	949
## theta[82]	-0.6117	0.2207	-1.0440	-0.6028	-0.2037	1.00	1125
## theta[83]	-0.2940	0.2886	-0.8842	-0.2889	0.2576	1.00	1115
## theta[84]	-0.6080	0.2268	-1.0662	-0.5892	-0.1702	1.01	1061
## theta[85]	-0.3364	0.2225	-0.7922	-0.3319	0.0692	1.00	1255
## theta[86]	0.0972	0.4815	-0.8689	0.1093	0.9902	1.00	997
## theta[87]	-0.2708	0.4172	-1.1588	-0.2706	0.5485	1.00	1210
## theta[88]	-0.1735	0.4356	-1.0312	-0.1813	0.7108	1.01	1059
## theta[89]	-0.0126	0.3438	-0.6878	-0.0070	0.6459	1.00	1583
## theta[90]	0.0209	0.2727	-0.5114	0.0164	0.5849	1.00	1000
## theta[91]	0.1493	0.4488	-0.7052	0.1464	0.9958	1.00	1008
## theta[92]	-0.4184	0.3591	-1.1045	-0.4061	0.2688	1.00	1097
## theta[93]	-0.0206	0.2610	-0.5139	-0.0247	0.4986	1.00	1000
## theta[94]	0.3193	0.2674	-0.2352	0.3242	0.8299	1.01	1089
## theta[95]	0.0570	0.4527	-0.7873	0.0513	1.0237	1.00	1004
## theta[96]	0.0364	0.2862	-0.5134	0.0336	0.6151	1.00	1114
## theta[97]	0.1864	0.3629	-0.5111	0.1844	0.8841	1.00	991
## theta[98]	0.4876	0.2334	0.0416	0.4864	0.9516	1.00	846
## theta[99]	0.6666	0.5080	-0.3362	0.6707	1.6763	1.00	1089
## theta[100]	0.9155	0.2674	0.4189	0.8977	1.4847	1.00	1227
## theta[101]	0.1262	0.2783	-0.4100	0.1254	0.6958	1.00	1000

```
## theta[102]    0.7779 0.4381 -0.0641  0.7606  1.6501 1.00  1000
```

```
MCMCsummary(object = ordinal_results$samples, params = "rho",
             # exact = TRUE,
             # ISB = FALSE,
             round = 4)
```

```
##      mean      sd   2.5%   50%  97.5% Rhat n.eff
## rho 0.6339 0.1528 0.3316 0.6482 0.9019   1  1037
```

```
MCMCtrace(object = ordinal_results$samples,
           pdf = FALSE, # no export to PDF
           ind = TRUE, # separate density lines per chain
           Rhat = TRUE,
           n.eff = TRUE,
           params = "rho")
```



```
test <- ordinal_results$samples

which((MCMCsummary(object = test, params = "kappa", round = 4)[, 6]) > 1.02 | (MCMCsummary(object
→ = test, params = "kappa", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "beta_age", round = 4)[, 6]) > 1.02 |
→ (MCMCsummary(object = test, params = "beta_age", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "beta_edu", round = 4)[, 6]) > 1.02 |
→ (MCMCsummary(object = test, params = "beta_edu", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "theta", round = 4)[, 6]) > 1.02 | (MCMCsummary(object
→ = test, params = "theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "sd.theta", round = 4)[, 6]) > 1.02 |
→ (MCMCsummary(object = test, params = "sd.theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "rho", round = 4)[, 6]) > 1.02 | (MCMCsummary(object
→ = test, params = "rho", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
### Bernoulli (D+) model ###
```

```
round(bernoulli_results1$summary, 4)
```

```
##          mean      sd    2.5%    50%   97.5% Rhat n.eff
## beta_0      -1.4713 0.0794 -1.6355 -1.4722 -1.3183 1.01 1052
## beta_age[1]  0.0000 0.0000 0.0000 0.0000 0.0000 NaN    0
## beta_age[2]  0.6750 0.0874 0.5142 0.6742 0.8481 1.01 1045
## beta_age[3]  1.6856 0.0763 1.5370 1.6804 1.8386 1.01 1054
## beta_edu[1]  0.0000 0.0000 0.0000 0.0000 0.0000 NaN    0
## beta_edu[2] -0.5212 0.0680 -0.6562 -0.5202 -0.3930 1.00 1066
## beta_edu[3] -0.9997 0.0718 -1.1435 -0.9978 -0.8627 1.00 1000
## rho          0.5259 0.1827 0.1648 0.5275 0.8518 1.00 743
## sd.theta     0.5808 0.0793 0.4394 0.5811 0.7466 1.00 871
## theta[1]     0.2257 0.4662 -0.6940 0.2215 1.1839 1.00 1015
## theta[2]     -1.0155 0.3190 -1.6820 -0.9943 -0.4337 1.00 874
## theta[3]     -0.9120 0.3447 -1.6465 -0.8976 -0.2913 1.01 1000
## theta[4]     -0.2419 0.4480 -1.1941 -0.2340 0.5680 1.01 982
## theta[5]     -0.2737 0.2797 -0.8320 -0.2766 0.2475 1.01 1074
## theta[6]     -0.2555 0.2700 -0.8135 -0.2487 0.2550 1.01 974
## theta[7]     -0.2560 0.3717 -0.9941 -0.2652 0.5050 1.01 2560
## theta[8]     -0.3068 0.3529 -0.9969 -0.3035 0.3778 1.00 1000
## theta[9]     -0.3275 0.4324 -1.2606 -0.3222 0.5251 1.00 957
## theta[10]    -0.1693 0.5154 -1.2063 -0.1646 0.8120 1.00 1086
## theta[11]    -0.7929 0.3097 -1.3880 -0.7781 -0.1850 1.00 1000
## theta[12]    -0.1473 0.3535 -0.8562 -0.1529 0.5692 1.00 941
## theta[13]    -0.2412 0.3309 -0.8529 -0.2291 0.3808 1.02 1257
## theta[14]     0.1602 0.3329 -0.4805 0.1645 0.8121 1.02 996
## theta[15]     0.2297 0.3540 -0.4425 0.2193 0.9387 1.00 855
## theta[16]    -0.0792 0.5656 -1.1448 -0.0817 1.0656 1.00 1003
## theta[17]    -0.3024 0.3688 -1.0388 -0.2884 0.3700 1.01 981
## theta[18]     0.2855 0.3800 -0.4396 0.2890 1.0721 1.00 1097
## theta[19]     0.0068 0.5371 -1.0292 0.0195 1.0342 1.00 1208
```

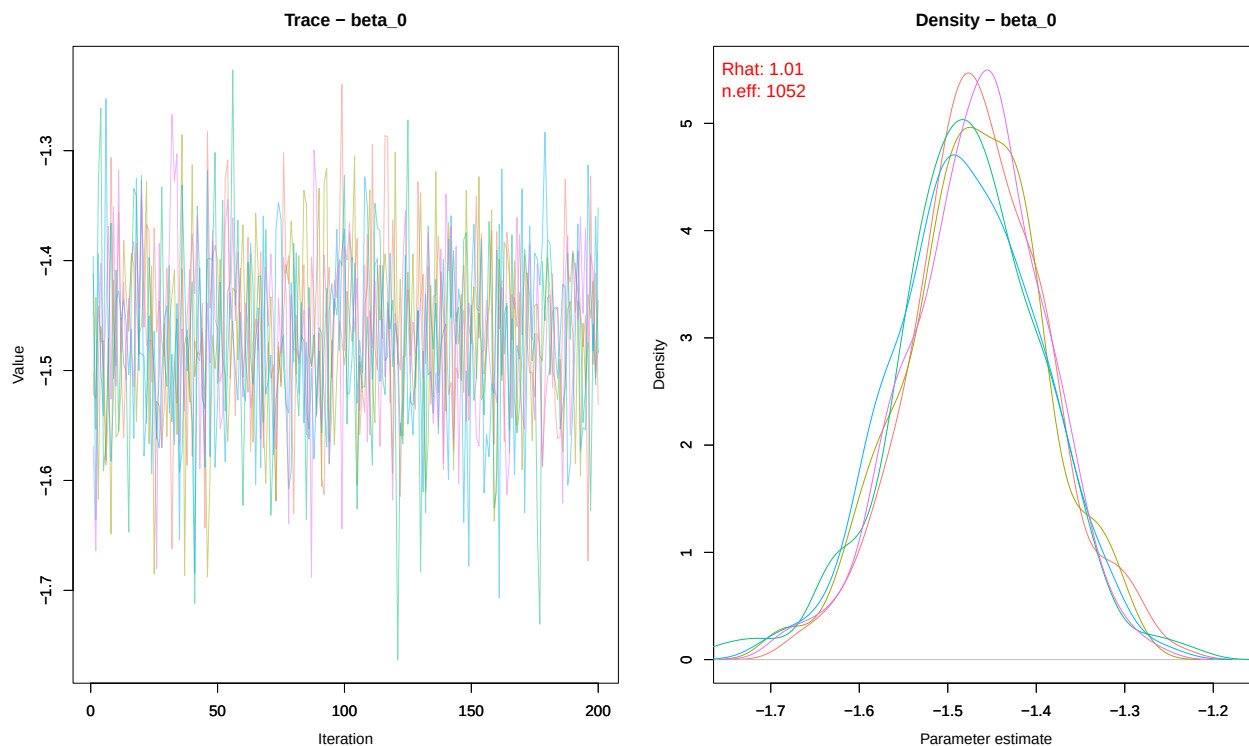
## theta[20]	-0.1700	0.4970	-1.1565	-0.1679	0.7960	1.02	1344
## theta[21]	-0.8612	0.3405	-1.5746	-0.8541	-0.2499	1.00	962
## theta[22]	-0.4802	0.2850	-1.0871	-0.4785	0.0393	1.00	1063
## theta[23]	-0.5720	0.3770	-1.3538	-0.5563	0.1340	1.01	884
## theta[24]	-0.4028	0.3811	-1.1022	-0.3986	0.3027	1.00	1000
## theta[25]	-0.8577	0.3467	-1.5467	-0.8447	-0.2068	1.00	1000
## theta[26]	-0.7418	0.4140	-1.6140	-0.7427	0.0480	1.00	1199
## theta[27]	-0.6845	0.5441	-1.7709	-0.6810	0.3750	1.01	1067
## theta[28]	1.0252	0.2793	0.5035	1.0059	1.6070	1.00	957
## theta[29]	0.7892	0.2654	0.3345	0.7722	1.3794	1.00	1036
## theta[30]	1.0309	0.5621	-0.0638	1.0287	2.1414	1.00	1000
## theta[31]	1.6047	0.4393	0.8028	1.5808	2.4716	1.00	1079
## theta[32]	0.4252	0.9863	-1.6240	0.4175	2.3601	1.00	1000
## theta[33]	-0.0470	0.6869	-1.4376	-0.0274	1.2387	1.01	1056
## theta[34]	0.8945	0.3298	0.2716	0.8993	1.5292	1.00	979
## theta[35]	0.8358	0.6290	-0.4006	0.8462	1.9801	1.00	1000
## theta[36]	0.8368	0.2732	0.3272	0.8219	1.4138	1.00	945
## theta[37]	0.9113	0.2201	0.5117	0.8944	1.3563	1.01	1000
## theta[38]	0.5875	0.4014	-0.2240	0.5826	1.3604	1.00	1170
## theta[39]	0.4860	0.7010	-0.8967	0.5004	1.7907	1.00	902
## theta[40]	0.7338	0.3831	-0.0058	0.7291	1.4718	1.00	1011
## theta[41]	1.5903	0.4699	0.6972	1.5780	2.5045	1.00	959
## theta[42]	0.4756	0.4500	-0.4063	0.4593	1.3805	1.00	1737
## theta[43]	1.1371	0.4639	0.2559	1.1248	2.1103	1.01	1186
## theta[44]	0.3340	0.3914	-0.4561	0.3404	1.1201	1.00	1175
## theta[45]	0.3050	0.6466	-1.0201	0.3121	1.5859	1.01	1197
## theta[46]	0.0453	0.6251	-1.2269	0.0539	1.2852	1.00	1000
## theta[47]	-0.1702	0.4582	-1.0492	-0.1610	0.7538	1.00	1064
## theta[48]	0.3970	0.5155	-0.6840	0.4021	1.4073	1.01	1047
## theta[49]	0.4364	0.6665	-0.8917	0.4514	1.6916	1.02	1272
## theta[50]	0.6948	0.4282	-0.1550	0.6888	1.5194	1.01	1011
## theta[51]	0.2993	0.3341	-0.3744	0.3053	0.9478	1.00	959
## theta[52]	0.4405	0.3118	-0.2124	0.4375	1.0620	1.00	1000
## theta[53]	0.0579	0.4275	-0.8011	0.0733	0.8795	1.00	1119
## theta[54]	0.0497	0.4729	-0.8707	0.0417	1.0103	1.00	1146
## theta[55]	0.1602	0.3189	-0.4605	0.1591	0.7749	1.01	1120
## theta[56]	0.6244	0.3433	-0.0022	0.6177	1.3044	1.00	1000
## theta[57]	0.6068	0.6173	-0.6430	0.5933	1.8496	1.01	1000
## theta[58]	0.1588	0.2898	-0.4096	0.1667	0.6992	1.00	1013
## theta[59]	0.4918	0.5265	-0.6005	0.4815	1.5358	1.00	1000
## theta[60]	-0.0557	0.3495	-0.7714	-0.0514	0.6393	1.01	1135
## theta[61]	0.8973	0.3806	0.1574	0.9027	1.6865	1.00	911
## theta[62]	0.7338	0.4395	-0.0846	0.7206	1.6334	1.01	922
## theta[63]	0.4667	0.4457	-0.3616	0.4623	1.2839	1.00	1053
## theta[64]	0.0339	0.5980	-1.1926	0.0296	1.1774	1.01	1174
## theta[65]	0.2447	0.5071	-0.7611	0.2406	1.2592	1.01	1086
## theta[66]	0.3443	0.4337	-0.5122	0.3437	1.1883	1.01	929
## theta[67]	0.2089	0.4795	-0.7581	0.2120	1.1502	1.00	1140
## theta[68]	0.8466	0.4668	-0.0510	0.8374	1.8518	1.01	1000
## theta[69]	0.7102	0.4635	-0.1617	0.7037	1.6136	1.01	1109
## theta[70]	0.9102	0.4206	0.0904	0.9069	1.7529	1.00	1052
## theta[71]	0.6942	0.4159	-0.1322	0.6983	1.5186	1.00	1000
## theta[72]	-0.1696	0.4582	-1.0889	-0.1479	0.7049	1.01	1060
## theta[73]	0.5875	0.4275	-0.2663	0.5845	1.3865	1.01	1000

```
## theta[74]    0.7838 0.5410 -0.2550  0.7638  1.8295 1.00 1121
## theta[75]    0.2930 0.5272 -0.8246  0.3054  1.2995 1.00 1298
## theta[76]    0.2074 0.4806 -0.7550  0.2028  1.1171 1.00 1384
## theta[77]    0.0285 0.4162 -0.7771  0.0205  0.8397 1.00 1261
## theta[78]    0.3863 0.4610 -0.5161  0.3869  1.3083 1.00 1352
## theta[79]    0.1500 0.3230 -0.4761  0.1459  0.7979 1.00 1040
## theta[80]   -0.0023 0.4599 -0.9283 -0.0147  0.8984 1.01  959
## theta[81]   -0.6024 0.1955 -1.0060 -0.5903 -0.2429 1.01  979
## theta[82]   -0.2704 0.2402 -0.7565 -0.2692  0.2195 1.01 1000
## theta[83]   -0.4514 0.3428 -1.1357 -0.4524  0.2166 1.00 1102
## theta[84]   -0.8876 0.2686 -1.4671 -0.8733 -0.3945 1.00 1000
## theta[85]    0.0233 0.2340 -0.4618  0.0268  0.4464 1.00  907
## theta[86]   -0.1633 0.5591 -1.2783 -0.1572  0.8729 1.00 1078
## theta[87]   -0.5611 0.4687 -1.4817 -0.5659  0.3517 1.00 1078
## theta[88]   -0.3763 0.4779 -1.3665 -0.3853  0.5478 1.00 1003
## theta[89]   -0.2631 0.3937 -1.0398 -0.2516  0.4921 1.00 1100
## theta[90]   -0.4023 0.3140 -1.0885 -0.3955  0.1994 1.00  905
## theta[91]   -0.3200 0.5214 -1.3391 -0.3334  0.7021 1.01 1052
## theta[92]   -0.6296 0.4317 -1.4729 -0.6141  0.2256 1.00 1017
## theta[93]   -0.1553 0.3054 -0.7906 -0.1417  0.4229 1.00 1647
## theta[94]   -0.0561 0.2952 -0.6213 -0.0580  0.5355 1.01 1000
## theta[95]   -0.4786 0.4729 -1.3700 -0.4705  0.4117 1.00 1036
## theta[96]   -0.3846 0.3398 -1.0892 -0.3951  0.2825 1.01 1202
## theta[97]   -0.0600 0.4131 -0.8780 -0.0682  0.7186 1.00 1198
## theta[98]    0.5910 0.2553  0.1045  0.5786  1.1012 1.00 1093
## theta[99]   -0.0571 0.6319 -1.3047 -0.0695  1.1954 1.00 1000
## theta[100]   1.0404 0.2970  0.5014  1.0253  1.6830 1.00  966
## theta[101]   0.2014 0.3058 -0.4070  0.1947  0.7932 1.01  962
## theta[102]   0.6828 0.4860 -0.2110  0.6946  1.6646 1.00 1023
```

```
MCMCsummary(object = bernoulli_results1$samples, params = "rho",
             # exact = TRUE,
             # ISB = FALSE,
             round = 4)
```

```
##      mean      sd  2.5%   50%  97.5% Rhat n.eff
## rho 0.5259 0.1827 0.1648 0.5275 0.8518    1   743
```

```
MCMCtrace(object = bernoulli_results1$samples,
           pdf = FALSE, # no export to PDF
           ind = TRUE, # separate density lines per chain
           Rhat = TRUE,
           n.eff = TRUE,
           params = "beta_0")
```

```
test <- bernoulli_results1$samples
```

```
which((MCMCsummary(object = test, params = "beta_0", round = 4)[, 6]) > 1.02 |  
  ↪ (MCMCsummary(object = test, params = "beta_0", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "beta_age", round = 4)[, 6]) > 1.02 |  
  ↪ (MCMCsummary(object = test, params = "beta_age", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "beta_edu", round = 4)[, 6]) > 1.02 |  
  ↪ (MCMCsummary(object = test, params = "beta_edu", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "theta", round = 4)[, 6]) > 1.02 | (MCMCsummary(object  
  ↪ = test, params = "theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "sd.theta", round = 4)[, 6]) > 1.02 |  
  ↪ (MCMCsummary(object = test, params = "sd.theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "rho", round = 4)[, 6]) > 1.02 | (MCMCsummary(object  
  ↪ = test, params = "rho", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
### Bernoulli (D-) model ###
```

```
round(bernoulli_results2$summary, 4)
```

##	mean	sd	2.5%	50%	97.5%	Rhat	n.eff
## beta_0	-3.4481	0.1811	-3.7970	-3.4525	-3.1059	1.00	843
## beta_age[1]	0.0000	0.0000	0.0000	0.0000	0.0000	NaN	0
## beta_age[2]	0.7014	0.1985	0.2953	0.6935	1.0706	1.01	732
## beta_age[3]	1.7697	0.1754	1.4279	1.7602	2.1191	1.00	948
## beta_edu[1]	0.0000	0.0000	0.0000	0.0000	0.0000	NaN	0
## beta_edu[2]	-0.6625	0.1211	-0.8826	-0.6626	-0.4334	1.00	1351
## beta_edu[3]	-1.3223	0.1454	-1.6033	-1.3241	-1.0282	1.00	1015
## rho	0.5285	0.1865	0.1752	0.5232	0.8698	1.00	840
## sd.theta	0.7771	0.1179	0.5648	0.7724	1.0342	1.00	879
## theta[1]	-0.8293	0.6407	-2.1496	-0.8290	0.3845	1.00	942
## theta[2]	-1.0446	0.4408	-1.9144	-1.0238	-0.2182	1.00	1000
## theta[3]	0.3286	0.3960	-0.4718	0.3354	1.0749	1.00	1040
## theta[4]	-0.0575	0.5315	-1.1610	-0.0405	1.0186	1.01	955
## theta[5]	-0.4865	0.4047	-1.3463	-0.4608	0.2731	1.00	1198
## theta[6]	-0.4813	0.3937	-1.2697	-0.4669	0.2549	1.00	1002
## theta[7]	0.3027	0.4184	-0.5401	0.3026	1.0738	1.01	1109
## theta[8]	0.1814	0.4004	-0.6316	0.1971	0.9164	1.01	870
## theta[9]	0.4192	0.4832	-0.5211	0.4405	1.4011	1.00	956
## theta[10]	0.1196	0.6604	-1.2227	0.1136	1.3261	1.00	1000
## theta[11]	-0.4269	0.3938	-1.2537	-0.4296	0.3535	1.00	1044
## theta[12]	0.0421	0.4270	-0.8215	0.0532	0.8754	1.00	1220
## theta[13]	-0.1365	0.4066	-0.9488	-0.1280	0.6314	1.00	1000
## theta[14]	0.2985	0.3822	-0.4574	0.3057	1.0313	1.00	1026
## theta[15]	-0.1870	0.4950	-1.2368	-0.1892	0.7897	1.00	1014
## theta[16]	0.3152	0.6186	-0.8775	0.3221	1.5108	1.00	1072
## theta[17]	0.3389	0.4099	-0.4512	0.3298	1.1325	1.00	1256
## theta[18]	0.4506	0.4072	-0.3808	0.4399	1.2222	1.00	1195
## theta[19]	0.1798	0.5759	-0.9665	0.1970	1.2962	1.00	1000
## theta[20]	0.3498	0.5313	-0.7664	0.3655	1.3665	1.00	1016
## theta[21]	-0.3744	0.4197	-1.2201	-0.3636	0.4247	1.00	1218
## theta[22]	-0.5459	0.4138	-1.3758	-0.5467	0.2240	1.01	1125
## theta[23]	-0.2586	0.4946	-1.2640	-0.2540	0.7120	1.00	1116
## theta[24]	-0.0960	0.4885	-1.0860	-0.0842	0.7879	1.01	1000
## theta[25]	-0.2866	0.3910	-1.0912	-0.2697	0.4537	1.00	954
## theta[26]	-0.5420	0.4886	-1.4713	-0.5398	0.4043	1.00	965
## theta[27]	-0.5467	0.6370	-1.8124	-0.5295	0.7013	1.00	1129
## theta[28]	1.1509	0.3195	0.5731	1.1388	1.8269	1.01	1000
## theta[29]	1.1178	0.2922	0.5429	1.1076	1.7351	1.00	938
## theta[30]	0.8922	0.6514	-0.4329	0.9283	2.1361	1.01	923
## theta[31]	1.1518	0.4757	0.2717	1.1452	2.0906	1.00	1013
## theta[32]	0.3216	1.0476	-1.6732	0.3314	2.3534	1.01	928
## theta[33]	0.3856	0.7294	-1.0998	0.4000	1.7646	1.00	1158
## theta[34]	1.2736	0.3618	0.5843	1.2808	2.0148	1.00	1046
## theta[35]	0.8200	0.6634	-0.5452	0.8133	2.0504	1.00	1000
## theta[36]	0.6275	0.3184	-0.0544	0.6134	1.2651	1.00	1140
## theta[37]	0.8735	0.2614	0.3888	0.8659	1.4289	1.00	1000
## theta[38]	0.8164	0.4382	-0.0090	0.7872	1.7282	1.00	993
## theta[39]	-0.0204	0.7473	-1.4362	-0.0413	1.4560	1.00	830

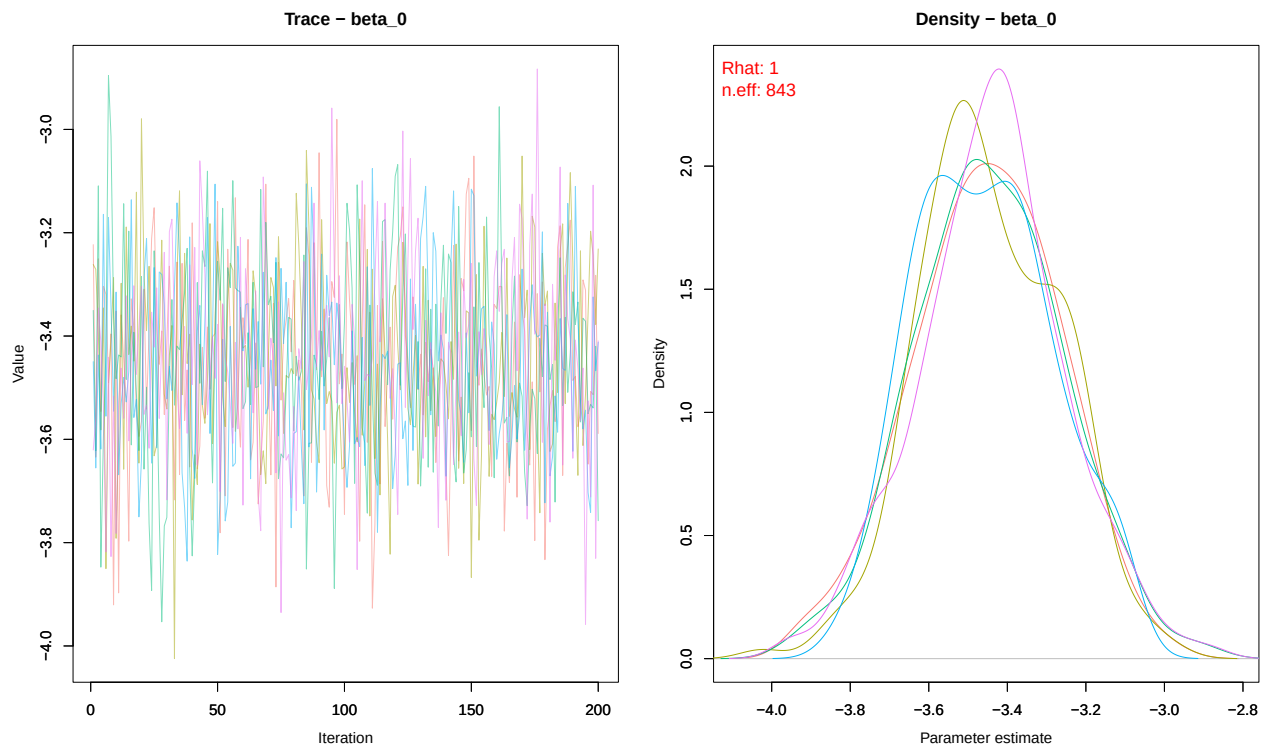
## theta[40]	1.0496	0.4415	0.1742	1.0624	1.8919	1.00	1216
## theta[41]	1.4862	0.4836	0.5817	1.4925	2.4380	1.00	1110
## theta[42]	0.1399	0.5769	-1.0070	0.1554	1.2394	1.00	1193
## theta[43]	1.6677	0.4768	0.7565	1.6773	2.6542	1.00	1072
## theta[44]	0.3997	0.4441	-0.4773	0.3978	1.2891	1.01	1000
## theta[45]	0.0274	0.7216	-1.4084	0.0436	1.4042	1.00	1000
## theta[46]	0.3573	0.6541	-0.9142	0.3588	1.6466	1.00	1561
## theta[47]	-0.0334	0.5245	-1.0408	-0.0254	0.9597	1.01	960
## theta[48]	0.3036	0.5658	-0.8422	0.3050	1.3770	1.01	956
## theta[49]	0.1296	0.6832	-1.2389	0.1311	1.4655	1.02	1030
## theta[50]	0.4329	0.4628	-0.4970	0.4228	1.3776	1.02	954
## theta[51]	-0.1711	0.4883	-1.2900	-0.1418	0.7391	1.01	1099
## theta[52]	0.3012	0.3558	-0.4060	0.3003	1.0031	1.00	1000
## theta[53]	-0.0141	0.4550	-0.9375	0.0209	0.8453	1.01	1074
## theta[54]	0.6699	0.4842	-0.2534	0.6750	1.6652	1.00	1131
## theta[55]	-0.2364	0.4011	-1.0398	-0.2221	0.5152	1.01	847
## theta[56]	0.0595	0.4024	-0.7213	0.0703	0.7891	1.00	1000
## theta[57]	-0.4503	0.7259	-1.9334	-0.4247	0.8697	1.01	943
## theta[58]	0.4190	0.3397	-0.2579	0.4141	1.0664	1.00	1031
## theta[59]	0.4258	0.5662	-0.7042	0.4444	1.5206	1.00	946
## theta[60]	0.5285	0.3908	-0.2248	0.5352	1.2997	1.00	1000
## theta[61]	0.0362	0.4627	-0.8912	0.0524	0.9533	1.00	1047
## theta[62]	0.3547	0.4933	-0.5997	0.3643	1.3430	1.00	1778
## theta[63]	0.0899	0.4998	-0.8697	0.0958	1.0712	1.00	1132
## theta[64]	-0.1514	0.6398	-1.4139	-0.1408	1.1238	1.01	1056
## theta[65]	0.6703	0.5425	-0.3585	0.6602	1.7102	1.01	969
## theta[66]	0.6993	0.4843	-0.3325	0.7115	1.5991	1.00	957
## theta[67]	0.4697	0.5368	-0.6095	0.4744	1.4859	1.00	1058
## theta[68]	0.3598	0.5104	-0.6854	0.3686	1.2715	1.00	1166
## theta[69]	0.3725	0.4550	-0.5855	0.3787	1.2424	1.00	1056
## theta[70]	0.2549	0.4915	-0.7042	0.2589	1.2051	1.01	1000
## theta[71]	0.2755	0.4897	-0.7232	0.3048	1.1967	1.00	1017
## theta[72]	-0.5222	0.6185	-1.7841	-0.5148	0.6388	1.00	1043
## theta[73]	-0.1340	0.5026	-1.1985	-0.1428	0.8744	1.00	1102
## theta[74]	0.3238	0.5657	-0.7442	0.3027	1.4904	1.00	1000
## theta[75]	-0.0473	0.5957	-1.4318	-0.0318	1.0390	1.00	1000
## theta[76]	0.2083	0.5139	-0.7675	0.2283	1.1904	1.00	958
## theta[77]	-0.1059	0.4571	-0.9996	-0.1001	0.7254	1.01	1000
## theta[78]	0.0107	0.5125	-1.0021	0.0025	1.0305	1.00	1107
## theta[79]	0.0674	0.3865	-0.6862	0.0704	0.8493	1.00	1126
## theta[80]	0.3563	0.5195	-0.7224	0.3797	1.3042	1.00	1531
## theta[81]	-0.6913	0.2865	-1.2768	-0.6808	-0.1675	1.00	1006
## theta[82]	-0.3600	0.3228	-1.0260	-0.3335	0.2499	1.00	1000
## theta[83]	-0.8193	0.5242	-2.0328	-0.7649	0.0705	1.00	1010
## theta[84]	-0.9182	0.3622	-1.6906	-0.8993	-0.2813	1.00	955
## theta[85]	-0.3664	0.3360	-1.0850	-0.3576	0.2568	1.01	1000
## theta[86]	-0.3130	0.6531	-1.6242	-0.3352	0.9130	1.01	1050
## theta[87]	-0.0846	0.5153	-1.0589	-0.0812	0.9130	1.00	954
## theta[88]	-0.0079	0.5433	-1.1515	0.0093	1.0380	1.01	1066
## theta[89]	-0.2217	0.4743	-1.1635	-0.2108	0.6568	1.00	1207
## theta[90]	0.0916	0.3736	-0.6680	0.0955	0.8295	1.01	1000
## theta[91]	-0.2683	0.5553	-1.3812	-0.2753	0.8539	1.00	1065
## theta[92]	-0.3365	0.5262	-1.4463	-0.3292	0.6173	1.01	1046
## theta[93]	-0.0577	0.4103	-0.9029	-0.0311	0.6771	1.00	1000

```
## theta[94] -0.0324 0.3633 -0.7649 -0.0189 0.6387 1.00 1067
## theta[95] -0.0554 0.5408 -1.1108 -0.0579 1.0539 1.00 1161
## theta[96] -0.1102 0.4039 -0.9549 -0.1084 0.6928 1.00 969
## theta[97] -0.0741 0.4684 -1.0179 -0.0499 0.8348 1.01 1093
## theta[98] -0.1566 0.3206 -0.8346 -0.1527 0.4272 1.01 1064
## theta[99] -0.3195 0.6932 -1.6784 -0.3087 0.9829 1.00 1000
## theta[100] -0.0722 0.3346 -0.7507 -0.0643 0.5685 1.00 1005
## theta[101] 0.1070 0.3707 -0.6485 0.1276 0.7876 1.01 1151
## theta[102] 0.5827 0.4665 -0.3532 0.5837 1.4723 1.00 1087
```

```
MCMCsummary(object = bernoulli_results2$samples, params = "rho",
  # exact = TRUE,
  # ISB = FALSE,
  round = 4)
```

```
##      mean      sd   2.5%   50%  97.5% Rhat n.eff
## rho 0.5285 0.1865 0.1752 0.5232 0.8698    1   840
```

```
MCMCtrace(object = bernoulli_results2$samples,
  pdf = FALSE, # no export to PDF
  ind = TRUE, # separate density lines per chain
  Rhat = TRUE,
  n.eff = TRUE,
  params = "beta_0")
```



```
test <- bernoulli_results2$samples

which((MCMCsummary(object = test, params = "beta_0", round = 4)[, 6]) > 1.02 |
  → (MCMCsummary(object = test, params = "beta_0", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "beta_age", round = 4)[, 6]) > 1.02 |  
→ (MCMCsummary(object = test, params = "beta_age", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "beta_edu", round = 4)[, 6]) > 1.02 |  
→ (MCMCsummary(object = test, params = "beta_edu", round = 4)[, 7] < 400))
```

```
## [1] 1
```

```
which((MCMCsummary(object = test, params = "theta", round = 4)[, 6]) > 1.02 | (MCMCsummary(object  
→ = test, params = "theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "sd.theta", round = 4)[, 6]) > 1.02 |  
→ (MCMCsummary(object = test, params = "sd.theta", round = 4)[, 7] < 400))
```

```
## integer(0)
```

```
which((MCMCsummary(object = test, params = "rho", round = 4)[, 6]) > 1.02 | (MCMCsummary(object  
→ = test, params = "rho", round = 4)[, 7] < 400))
```

```
## integer(0)
```